

交通大数据

深度学习

- 刘志远教授
- zhiyuanl@seu.edu.cn



深度学习

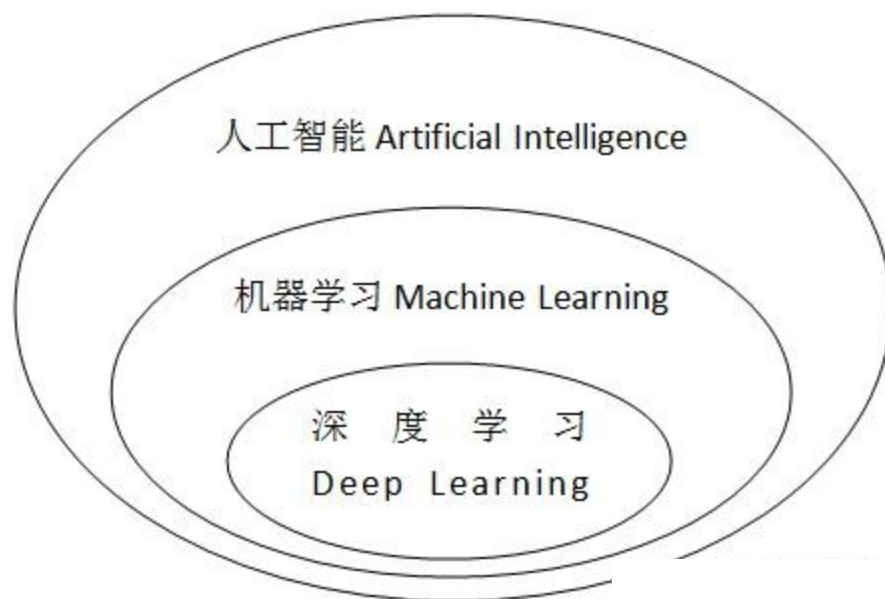
□ 学习目标

- 掌握卷积神经网络的基本原理
- 掌握循环神经网络的基本原理
- 了解图神经网络的基本概念
- 掌握应用简单的深度学习模型解决实际问题的方法



深度学习

深度学习首先是一种机器学习方法
深度学习的基础是神经网络模型。

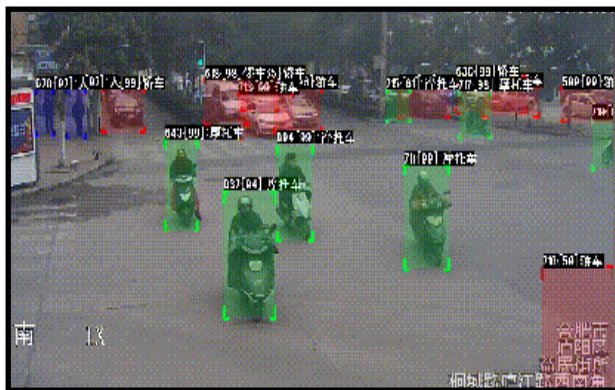


深度学习

为什么需要深度学习为交通问题建模

- ✓ 交通大数据存在大量半结构化与结构化数据，如车辆轨迹、城市道路过车记录、视频数据等等。它们往往具有复杂的时空特征，一方面我们很难将其整理成结构化的输入供经典机器学习模型进行训练，另一方面经典模型也难以在这些任务上达到期望的精度。

道路交通流分析



公交客流统计和OD识别



道路交通事件识别



深度学习

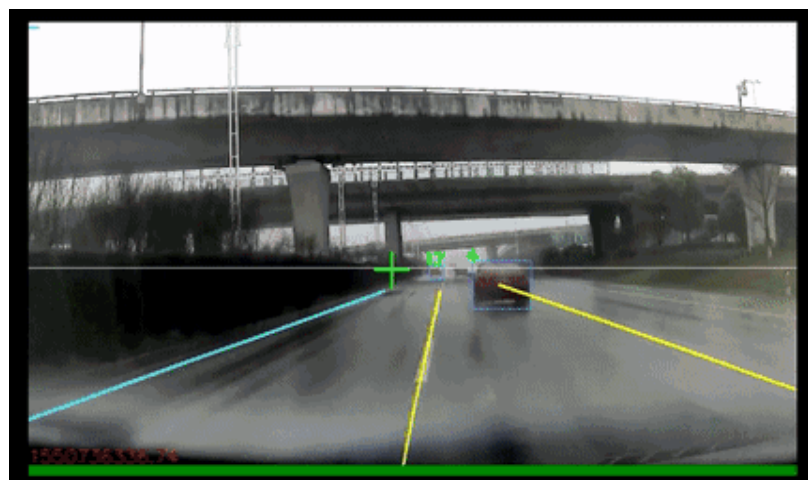
为什么需要深度学习为交通问题建模

- ✓ 想要准确地对各类数据间的非线性关系建模，就需要假设空间更大、复杂度更高的模型。在第11章中，我们介绍过神经网络可以作为一种万能近似器，拟合各类复杂的非线性映射。在此基础上，我们很容易想到，通过增加神经网络中神经元的数量或隐藏层层数，可以大大提升模型的性能。

自动驾驶环境感知



微观驾驶行为分析

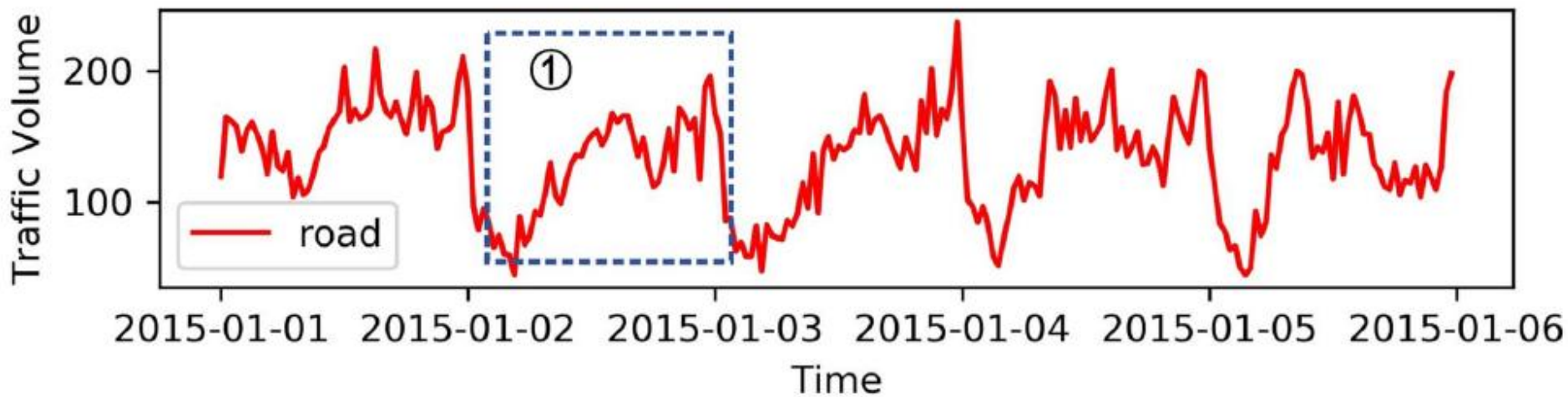


- ✓受制于计算能力和数据资源，复杂的神经网络模型难以被充分地训练，且容易出现过拟合的问题。2010年前后，随着计算能力的迅猛提升和大数据的涌现，训练拥有海量参数的模型成为可能，神经网络研究在“深度学习”的名义下又重新崛起，在计算机视觉、语音识别、自然语言处理等诸多重要领域和任务上都取得了显著的性能突破。



深度学习

- ✓ 尽管作为一门新兴技术，深度学习在近几年才广为人知，但是其中很多概念与理念在20世纪就已出现。除了第11章介绍的全连接神经网络外，深度学习两大最具代表性的结构是**卷积神经网络** (Convolutional Neural Networks, CNN) 和**循环神经网络** (Recurrent Neural Network, RNN)，也将是本章的重点内容。

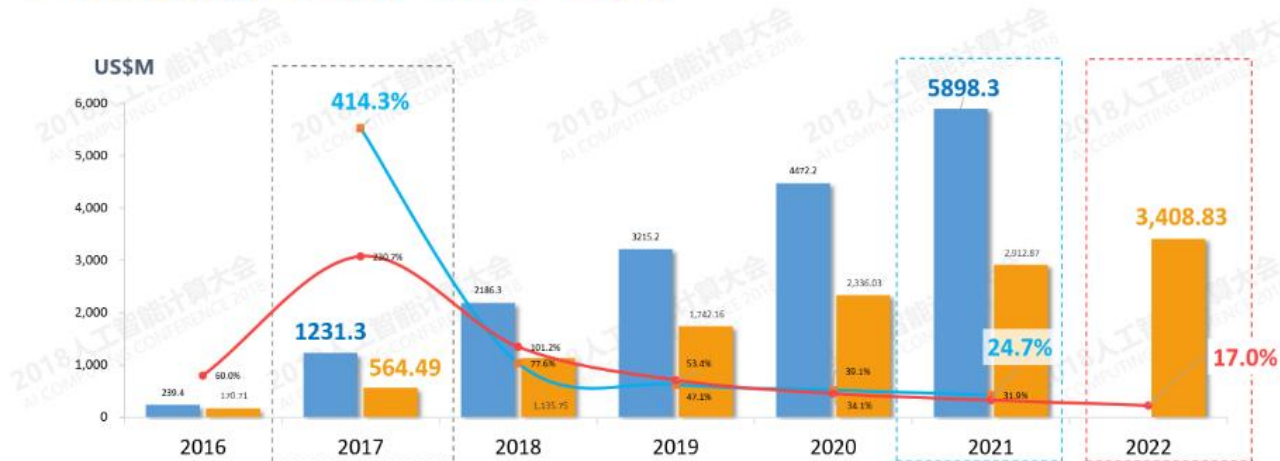


深度学习

算力与深度学习

- ✓ 深度学习模型在大数据量以及大算力的条件下才能发挥其优势，而早期深度学习模型的发展严重收到硬件条件的限制。近十年，人类已经进入大数据时代，硬件（CPU/GPU/TPU/NPU）也得到了飞速的发展，算力得到了有效保障。

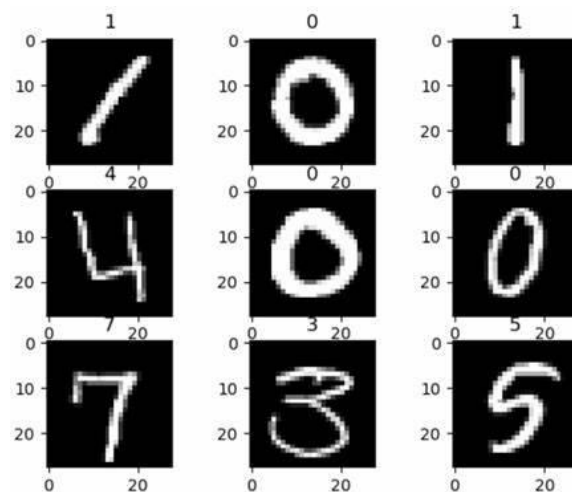
AI击鼓扬帆正当时 投资 算力



卷积神经网络 CNN

□ 发展历程

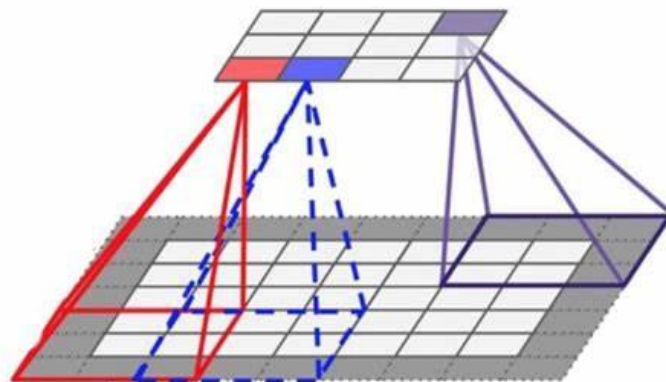
- 1962年，Hubel和Wiesel在研究猫脑皮层中用于局部敏感和方向选择的神经元时发现其独特的局部互连网络结构可以有效地降低反馈神经网络的复杂性，继而提出了卷积神经网络。
- 1989年，LeCun首次将CNN应用在计算机视觉分析中，用于手写数字和字母的识别。
- 2012年，Ciresan等和Krizhevsky等先后建立了基于CNN的深度神经网络，并在ImageNet图像识别挑战赛中展现出大幅超越传统模型的性能。



卷积神经网络 CNN

- 在视觉神经系统中，一个神经元的**感受野**是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。
- 卷积神经网络通常适用于网格数据（定义在欧几里得空间中的数据）的处理，既可以是较为直观的二维网格，也可以是一维、三维或更高维的网格。

感受野 (Receptive Field)



卷积神经网络 CNN

- 我们通常所说的卷积神经网络是指“至少使用一个卷积层”的神经网络模型。
- 神经网络卷积的计算方法与数学上的卷积运算类似，用其代替普通全连接层的矩阵乘法运算。
- 卷积神经网络在结构上有三个特点：
 - 局部连接 (sparse connectivity)
 - 参数共享 (parameter sharing)
 - 下采样 (subsampling)

这些特性使得卷积神经网络具有一定程度上的平移不变性(translation invariant)。



卷积神经网络 CNN

□ 数学中的卷积运算 convolution

卷积是分析数学中一种重要的运算。对于两个函数 f 与 g 的卷积，可以看作它们经过翻转和平移后**重叠部分的面积**。我们记卷积运算符号 $*$ ，则函数 f 和 g 的卷积为 $f * g$ ：

$$(f * g)(x) = \int_{-\infty}^{\infty} f(t)g(x - t)dt$$

上式为卷积定义于实数域的连续函数形式，考虑到输入计算机的数据是包含**有限个元素**的离散值，则卷积计算可表示为：

$$(f * g)(x) = \sum_{-\infty}^{\infty} f(t)g(x - t)$$



卷积神经网络 CNN

□ 神经网络的卷积运算 convolution

$$(f * g)(x) = \sum_{-\infty}^{\infty} f(t)g(x - t)$$

在卷积神经网络中， f 为**输入** (input)， g 被称为**卷积核** (convolution kernel)。其输出则被称为特征映射 (feature map)，特征映射能够将数据张量映射到特征空间中。由于神经网络中的输入与卷积核均为大小有限的张量，运算结果实际上是两个张量间**哈达玛积*的各元素之和**。

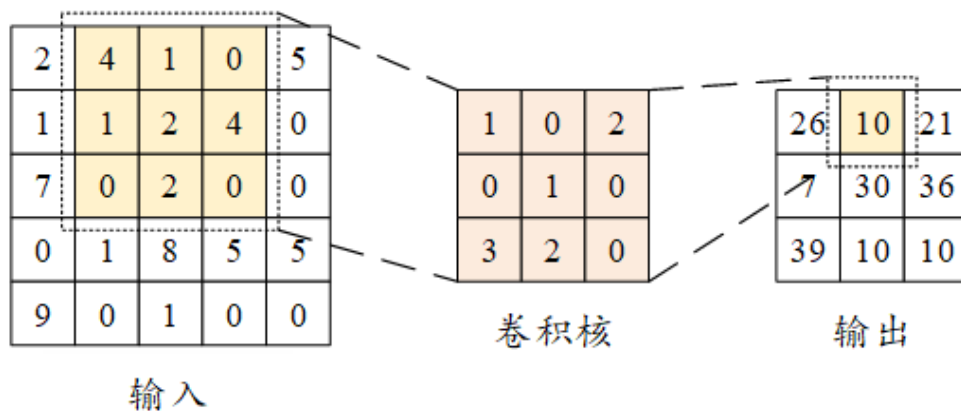
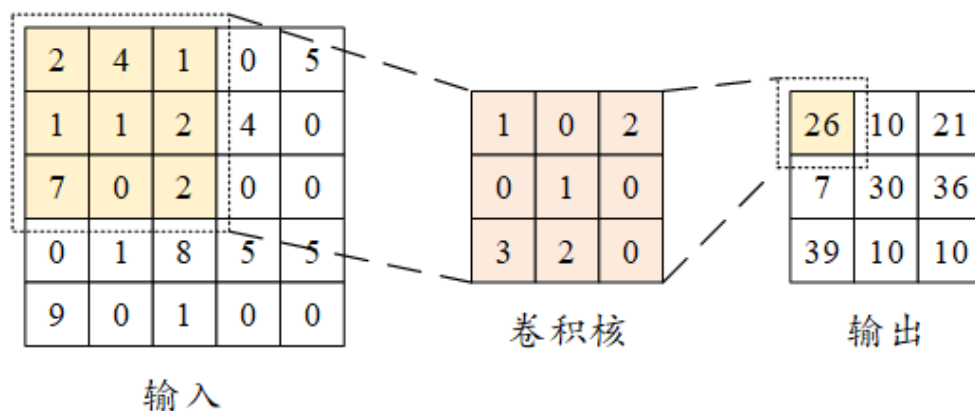
*两相同尺寸张量(tensor)间的**哈达玛积**，是将这两个张量逐元素相乘。

$$\begin{array}{|c|c|} \hline 2 & 3 \\ \hline 0 & 1 \\ \hline \end{array} \otimes \begin{array}{|c|c|} \hline 3 & 1 \\ \hline 2 & 4 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 6 & 3 \\ \hline 0 & 4 \\ \hline \end{array}$$



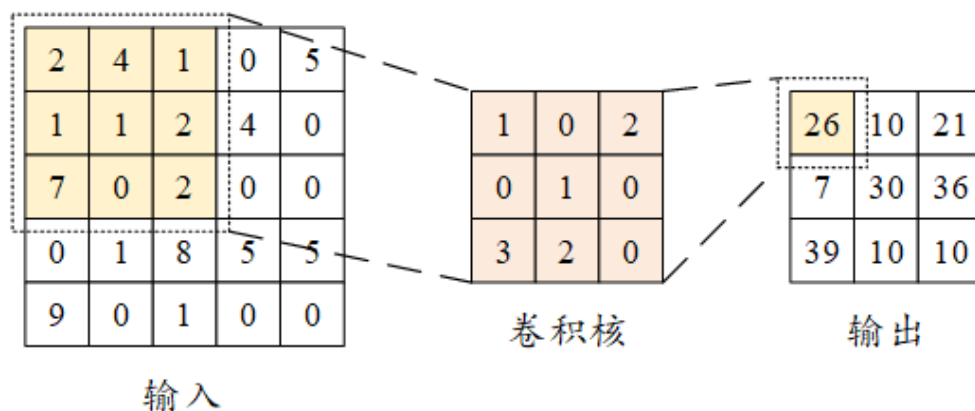
神经网络卷积运算 convolution

“两个张量间哈达玛积的各元素之和”



□ 神经网络卷积运算 convolution

“两个张量间哈达玛积的各元素之和”



此处的卷积运算，需要先将卷积核顺时针旋转180度（即卷积核上下翻转、再左右翻转），然后再做乘积。但为了方便计算，在深度学习中，通常不进行旋转，而是直接做乘积，即“**互相关运算**” (cross-correlation)。两种运算在深度学习中是等价的，是因为深度学习中的卷积核是学出来的，因此这个旋转操作就不产生影响了。因此大多数深度学习中提到的卷积运算均为互相关运算。

试一试

基于numpy实现卷积的计算

```
import numpy as np
```

```
# 互相关运算（与卷积结果相同）
```

```
def conv_2d(inputs, kernel):
```

```
    # inputs和kernel都是二维张量
```

```
    pass
```

```
# 输入矩阵
```

```
input_arr = np.array([[2, 4, 1, 0, 5],  
                      [1, 1, 2, 4, 0],  
                      [7, 0, 2, 0, 0],  
                      [0, 1, 8, 5, 5],  
                      [9, 0, 1, 0, 0]])
```

```
# 卷积核
```

```
kernel = np.array([[1, 0, 2], [0, 1, 0], [3, 2, 0]])
```

```
# 进行运算
```

```
print(conv_2d(input_arr, kernel))
```



试一试

基于numpy实现卷积的计算

互相关运算（与卷积结果相同）

```
def conv_2d(inputs, kernel):
```

```
    iw, ih = inputs.shape
```

```
    kw, kh = kernel.shape
```

```
    outputs = np.zeros((iw - kw + 1, ih - kh + 1))
```

```
    for s in range(outputs.shape[0]):
```

```
        for t in range(outputs.shape[1]):
```

```
            outputs[s, t] = (inputs[s:s + kw, t:t + kh] * ker
```

```
nel).sum())
```

```
    return outputs
```

输出

```
[[26. 10. 21.]
```

```
 [ 7. 30. 36.]
```

```
 [39. 10. 10.]]
```



卷积神经网络 CNN

□ 神经网络卷积运算 convolution

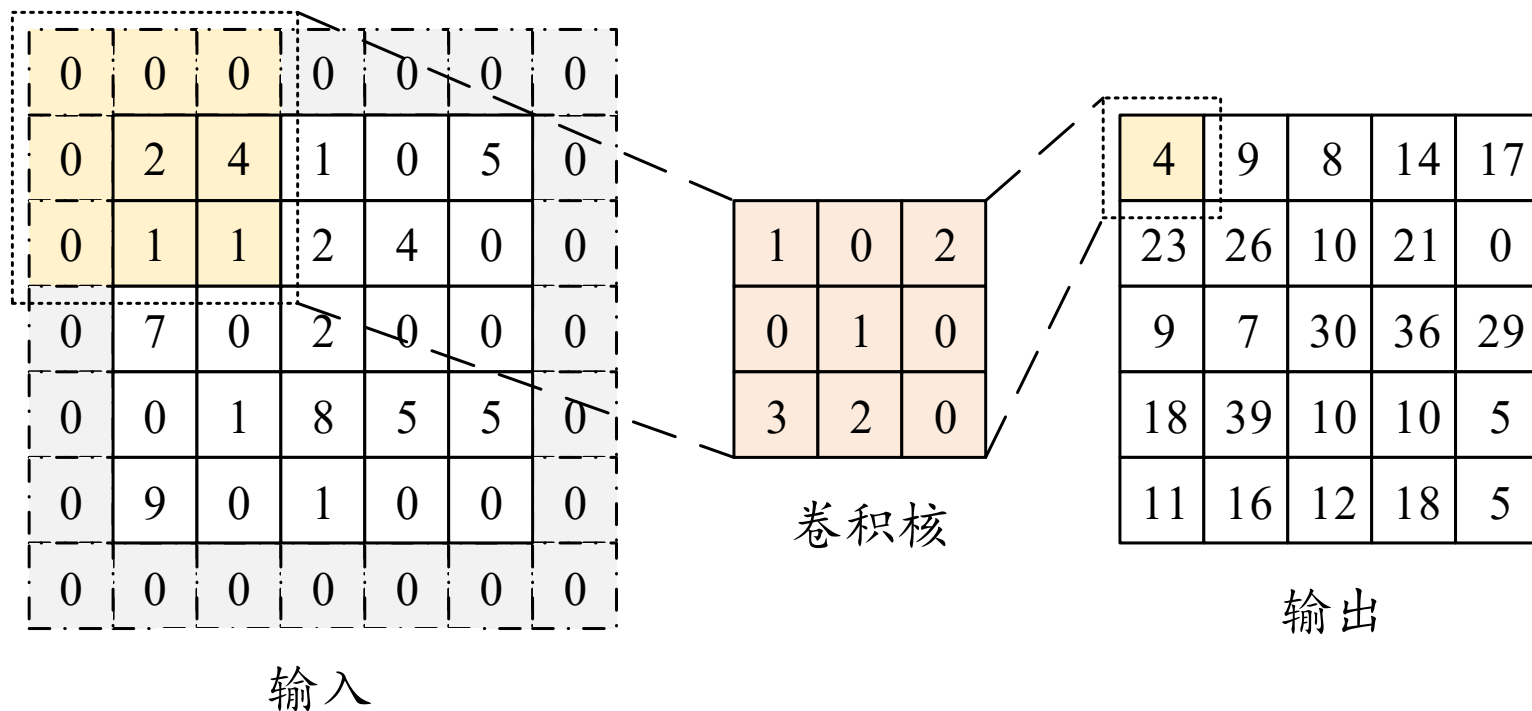
需要注意，实际应用中输入矩阵和卷积核的形状并**不一定**为方阵，输出矩阵的形状也**不一定**与卷积核相同。

我们将引入两个额外的参数：填充(padding)和步长(stride)，能影响输入矩阵的形状和卷积核的滑动方式，从而影响最终的特征映射大小



卷积神经网络 CNN

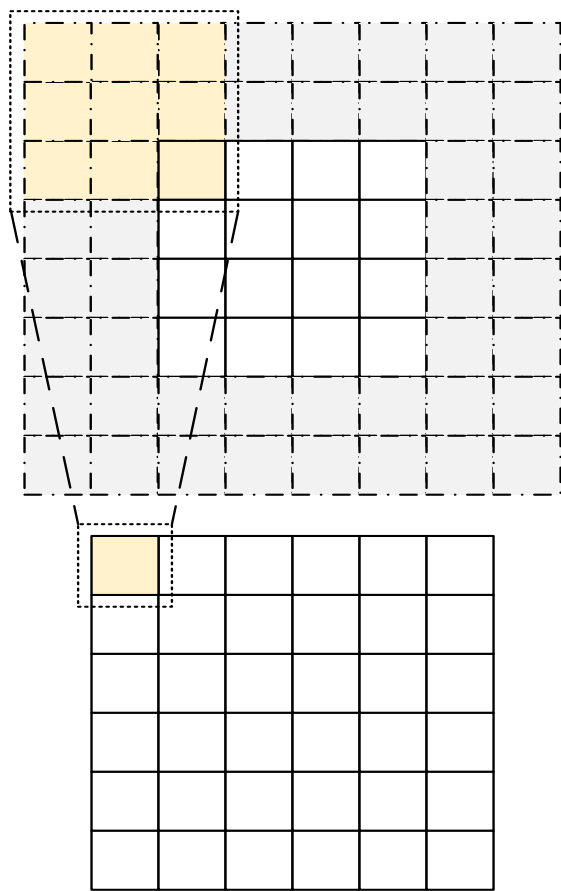
□ 填充padding



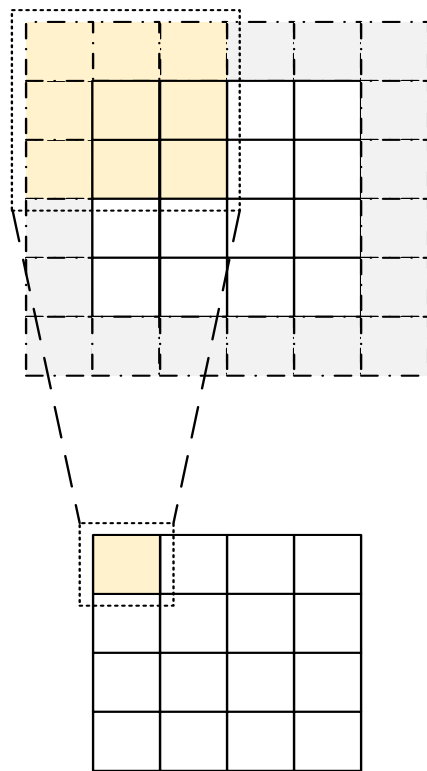
* 注：填充值不一定为0

卷积神经网络 CNN

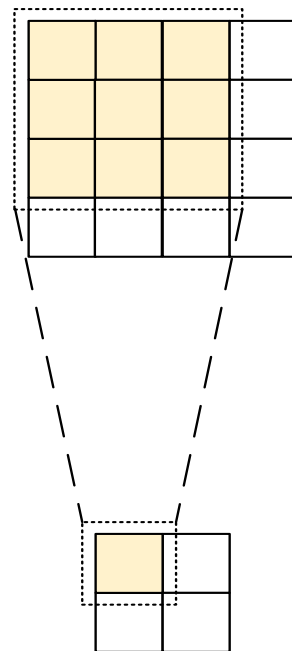
- 输入矩阵的填充不同，卷积核的移动范围就不同。根据卷积核的移动范围，我们将卷积分为三种模式：完全填充、相同填充、无填充。



完全填充



相同填充

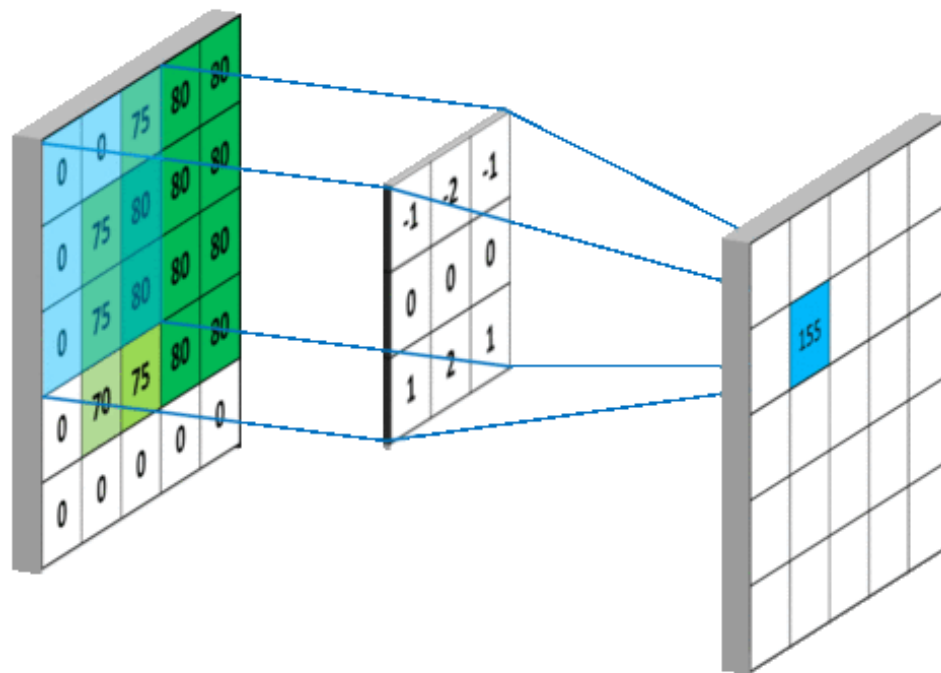


无填充

卷积神经网络 CNN

□ 步长stride

在做卷积运算时，也可以指定卷积核滑动的步长



卷积神经网络 CNN

□ 神经网络卷积运算 convolution

需要注意，实际应用中输入矩阵和卷积核的形状并**不一定**为方阵，输出矩阵的形状也**不一定**与卷积核相同。

此处介绍的两个额外参数：填充(padding)和步长(stride)，能影响输入矩阵的形状和卷积核的滑动方式，从而影响最终的特征映射大小

输出尺寸结论：(思考)

若输入张量的尺寸为 $i_h \times i_w$ ，卷积核的尺寸为 $k_h \times k_w$ ，在宽、高方向两侧各做长度为 p_h 和 p_w 的填充，在宽、高方向上移动步长分别为 s_h 和 s_w ，则输出的特征映射形状为？

$$\lfloor (i_h + 2p_h - k_h + s_h)/s_h \rfloor \times \lfloor (i_w + 2p_w - k_w + s_w)/s_w \rfloor$$

* $\lfloor \dots \rfloor$ 为向下取整符



卷积神经网络 CNN

□ 多通道卷积

在实际应用中，我们往往需要同时处理包含更多信息的数据。

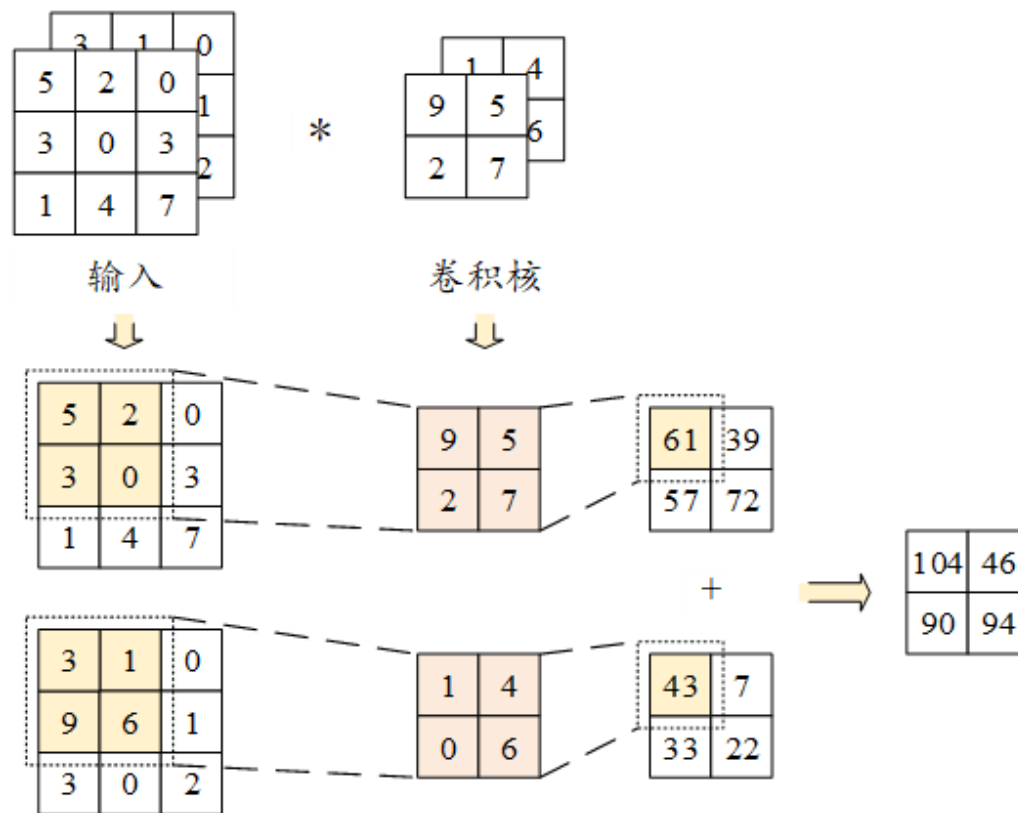
- ✓ 对于常见的彩色图片而言，每一个像素点都是由红、绿、蓝三原色组成的；（其包含三个通道、对应于三种原色）
- ✓ 在交通领域，如果将一个城市区域划分为多个小网格，那么每一个网格中都可能有多种数据，如各类兴趣点（POI）的数量、交通量等，像地图软件不同“图层”。（包含多个通道，对应于每种数据）

换言之，之前案例中的每个网格，其包含的不一定是一个单一的数值，也有可能是一个向量。我们称向量的每一维为一个**通道**（channel）。



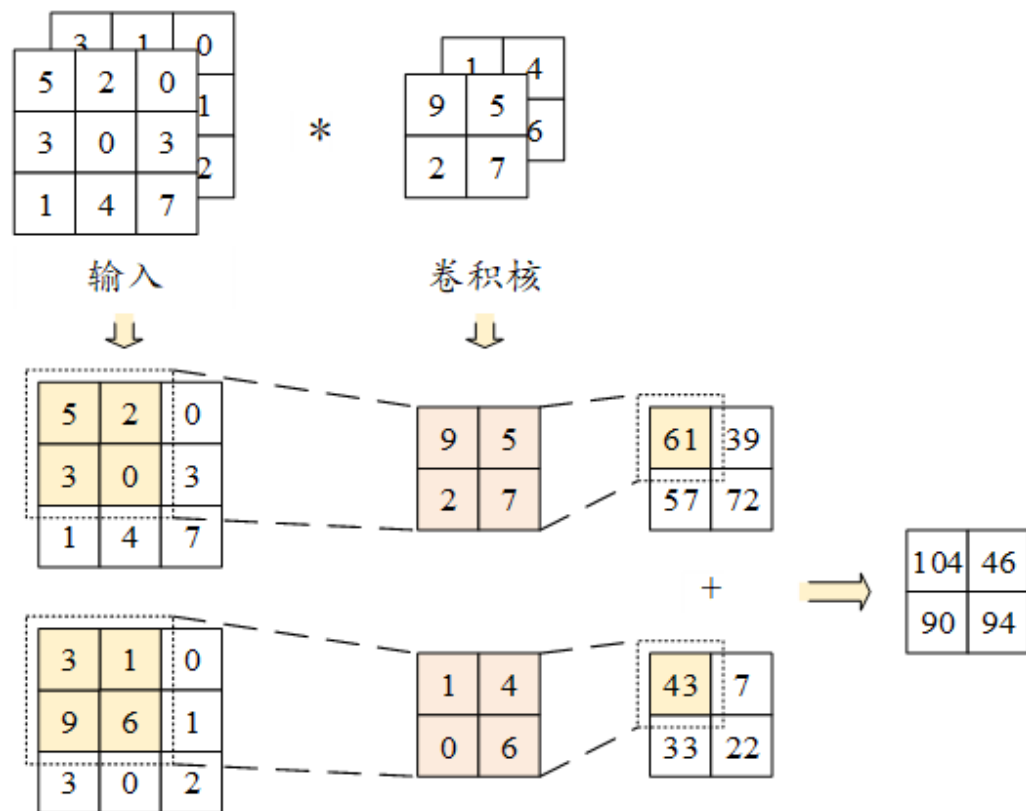
多通道卷积

由于一个二维的卷积核无法与多通道输入直接进行卷积运算，在对多通道输入进行卷积时，需要为卷积核也赋予**等量的通道**。卷积核的每一个通道与输入张量的每一个通道——对应。分别执行完卷积运算后，再将计算结果逐元素相加，即得到最终输出结果。



多通道卷积

不难发现，多通道输入经过卷积运算后，输出的结果只有一个通道。如果在一次卷积运算中，采用**有多个通道的卷积核**，输入张量的每个通道经过卷积核对应通道处理之后都能得到一个结果，最终输出张量就是将每个卷积核的结果沿着通道进行叠加。



试一试

基于numpy实现多通道卷积计算

```
import numpy as np

# 多通道卷积计算（padding与stride不做要求）
def conv_2d_channel(input_arr, kernel):
    pass

# 输入张量
input_arr = np.array([[[5, 2, 0],[3, 0, 3],[1, 4, 7]],
                      [[3, 1, 0], [9, 6, 1], [3, 0, 2]]])

# 卷积核
kernel = np.array([[[9, 5], [2, 7]],
                   [[1, 4], [0, 6]]])

# 进行运算
print(conv_2d_channel(input_arr, kernel))
```



试一试

基于numpy实现多通道卷积计算

```
import numpy as np

# 多通道卷积
def conv_2d_channel(input_arr, kernel):
    outputs = np.stack([
        conv_2d(arr, ker)
        for arr, ker in zip(input_arr, kernel)])
    return outputs.sum(axis=0)
```

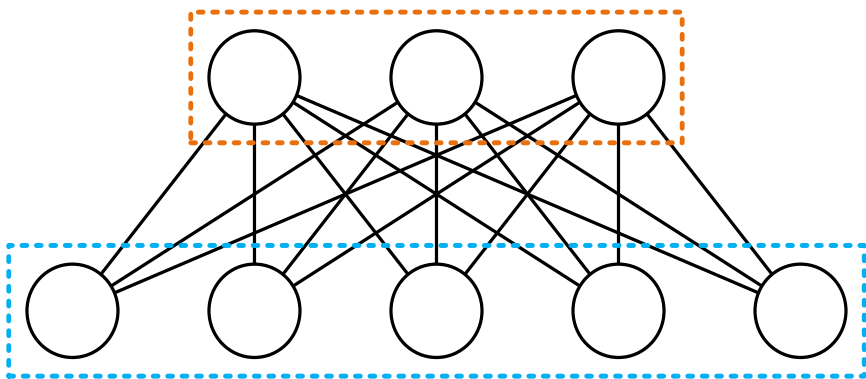
```
# 输出
[[104. 46.]
 [ 90. 94.]]
```



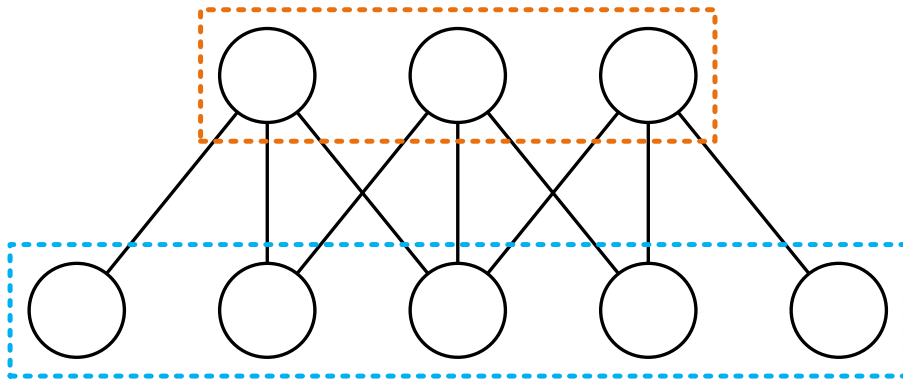
卷积神经网络 CNN

❑ 卷积层 convolution layer

(a) 全连接层



(b) 卷积层



(每个输入网格都可以看作是一个神经元)

卷积神经网络 CNN

❑ 卷积层 convolution layer

● 局部连接：

卷积神经网络下层的神经元，只能直接或间接地与原始输入数据中的**一部分**数据相连接。

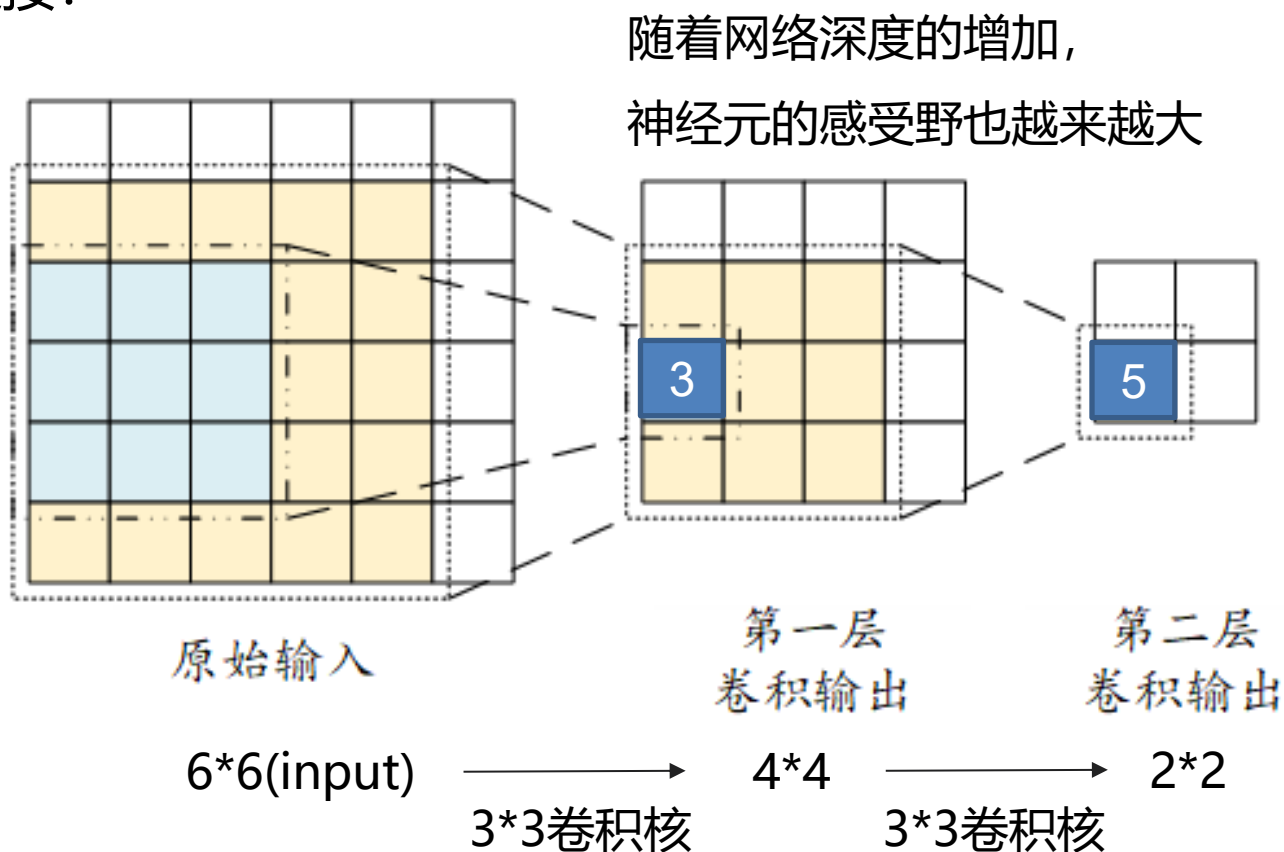
每一层输出的特征映射中每个神经元与**原始输入**间接连接的范围称为“感受野”。感受野更大的神经元有更大的机会获取更丰富的信息，感受野较小的神经元往往只能关注局部的细节特征。



卷积神经网络 CNN

□ 卷积层 convolution layer

● 局部连接:



卷积神经网络 CNN

□ 卷积层 convolution layer

● 参数共享:

为减少CNN的参数，在卷积层中使用参数共享的方法。在同一个卷积层的卷积计算中（同一个通道），卷积过程中卷积核的权重不会改变。

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

卷积神经网络 CNN

❑ 卷积层 convolution layer

● 参数共享：

- 对于一个卷积层中的所有神经元，权重都是相同的（同一个卷积核对于不同输入网格权重相同）。
- 例如，有一个 3×3 的卷积核，在一个 5×5 的输入图像上应用，这个卷积核会在图像的每个位置滑动，计算出一个 3×3 区域的加权和，**在每次滑动时，卷积核的参数（权重）保持不变，这就是参数共享。**
- 这种机制可以显著地减少需要学习的参数量，以及模型需要占用的内存；参数共享使得模型不会对输入的位移敏感，从而拥有一定的**平移不变性**。



卷积神经网络 CNN

❑ 卷积层 convolution layer

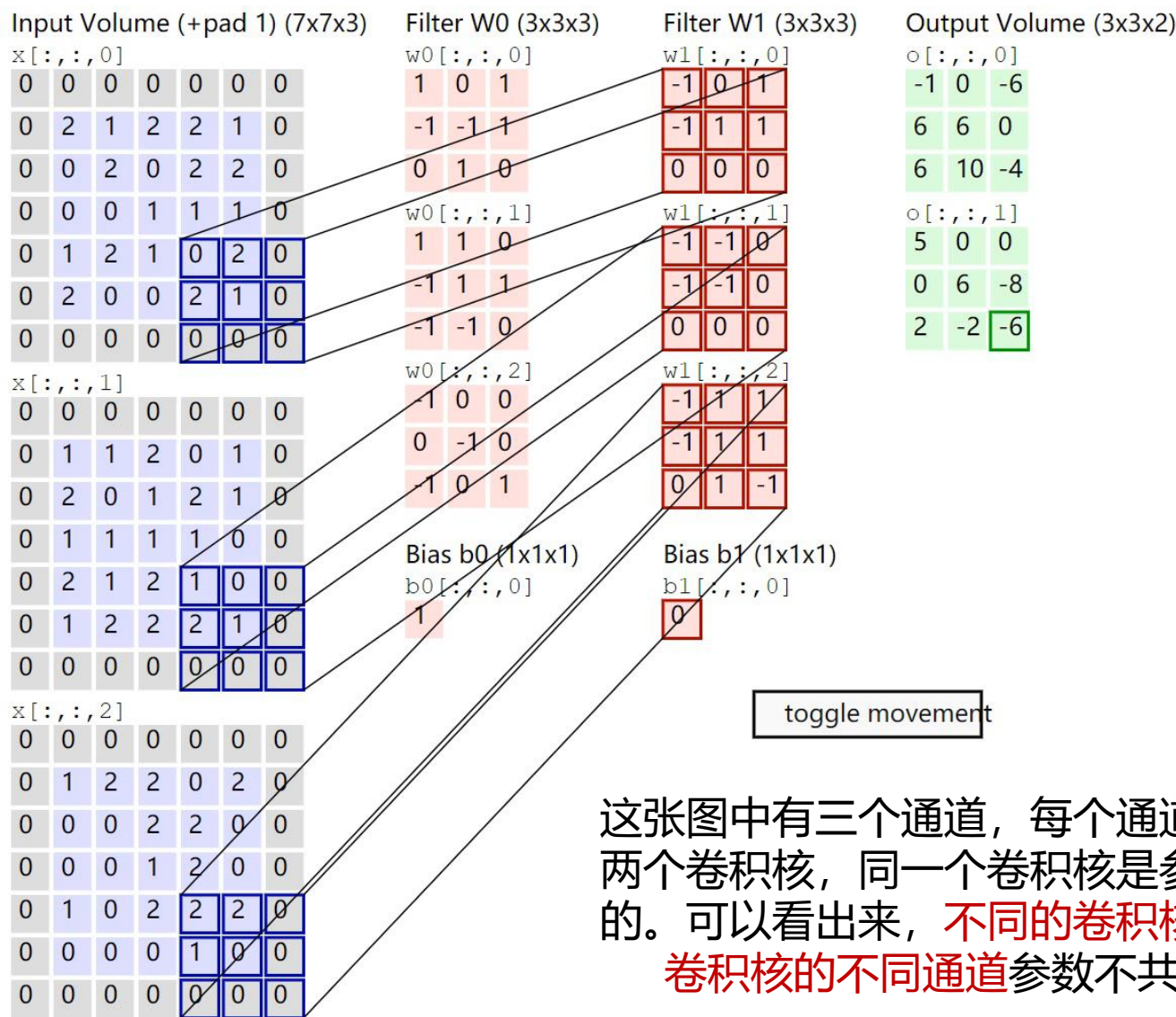
● 参数共享：

参数共享特性另外一个需要注意的点在于，卷积层中的参数共享仅限于同一个通道。若输入张量包含多个通道，则卷积核每个通道里的权重值是不同的。换言之，卷积神经网络在进行特征提取时，对于不同通道的特征是分开处理的（不同的通道参数不是共享的）。

由于一个卷积核只能提取一种模式的特征，即使对不同通道采用不同的权重，特征提取的限制仍然较大。因此，在实践中，常常（对同一个通道）同时使用多个卷积核，希望它们能够提取出不同类型的特征。不同卷积核之间的参数也是不共享的。



参数共享



卷积神经网络 CNN

□ 卷积层 convolution layer

● 平移不变性：

平移是一种几何变换，表示把一幅图像或一个空间中的每一个点沿相同方向移动相同距离。平移不变性在计算机视觉领域的应用中是一个十分重要的特性，比如对图像分类任务来说，图像中的目标不管被移动到图片的哪个位置，得到的结果（标签）应该是相同的，这就是卷积神经网络中的平移不变性。

当其应用在交通领域时，需要考虑这一特性是否符合相关的领域知识和工程实际。（例如交通网络中，两个离得很远的路段，如说设计指标一致，其交通状态不一定有关联性、相似性，因此不满足平移不变性。这种情况下，应该把位置坐标也作为通道输入模型，解决该问题。）

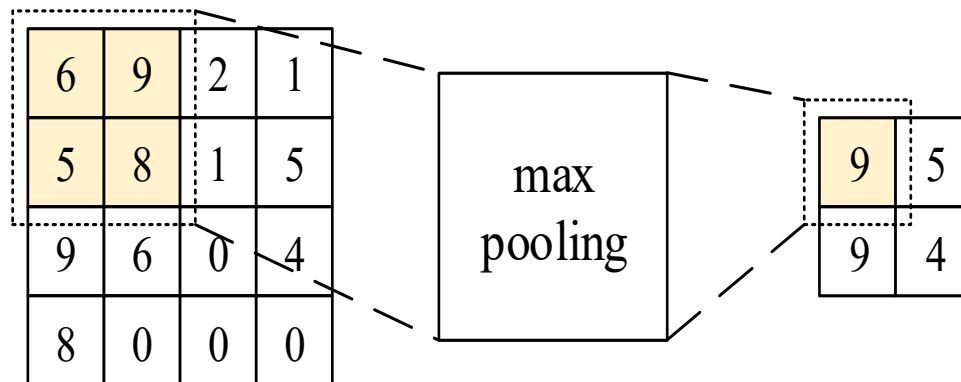


卷积神经网络 CNN

❑ 池化层 pooling layer

● 下采样down sampling

一些卷积核可能会将数据中的噪声视为某种特征，从而导致过拟合的问题，使得模型的泛化能力不足。为了避免噪声和微小扰动的影响，可以对数据进行**下采样**的操作，来降低模型对数据的敏感度。下采样的一种常用方法是**池化**，常用的方法有**最大值池化**（max pooling）和**平均值池化**（average pooling）。

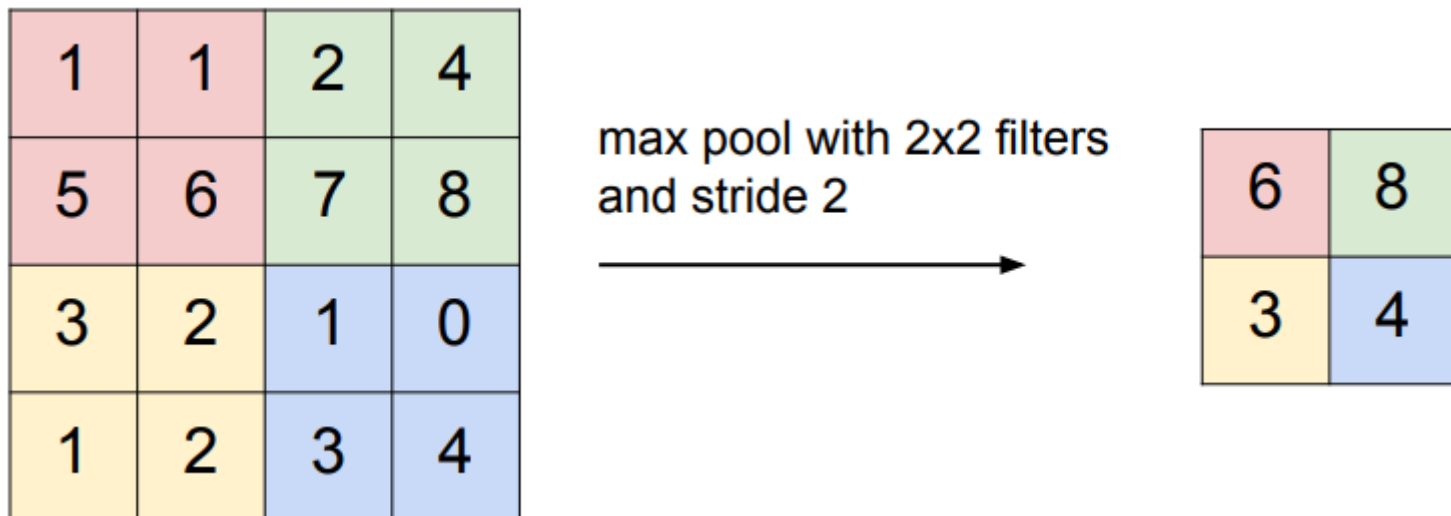


卷积神经网络 CNN

❑ 池化层 pooling layer

- 下采样down sampling

池化窗口的移动方式也和卷积核的滑动方式一样，同样具有填充和步长的概念，能够改变输出的尺寸。



卷积神经网络 CNN

❑ 池化层 pooling layer

- 下采样down sampling

池化层对于多通道的处理与卷积层是不一样的。卷积层中，对每个通道分别做卷积运算后，需要将各个结果逐元素相加，**因此一个卷积核只会**
有一个输出通道。而池化层中，对每个通道分别做池化运算后，不做任何
处理。换言之，**池化层的输出和输入的通道数是没有差异的**。

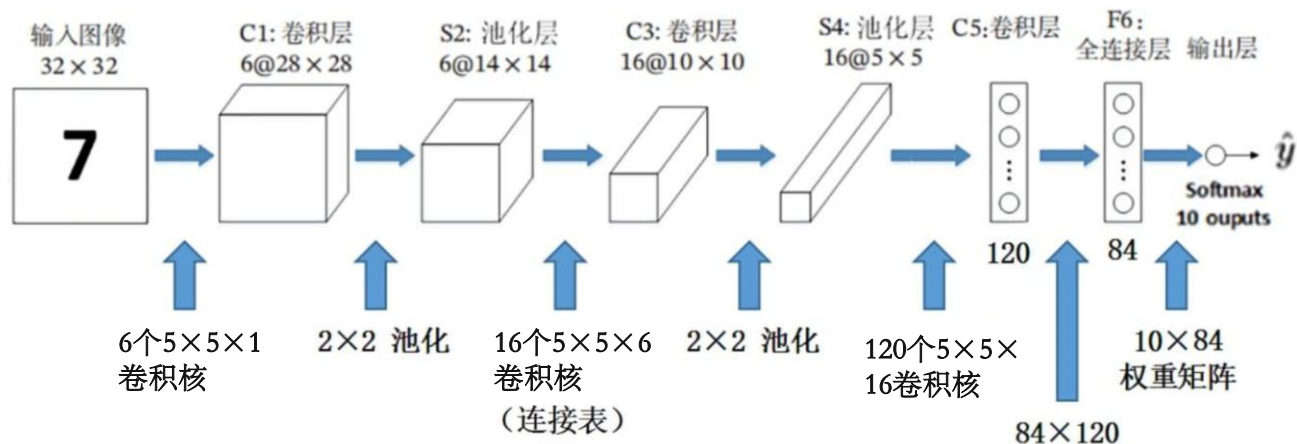
例如：对于input shape为(5,5,**3**)的张量，经过padding=0, stride=2
的2*2的池化层运算后，输出的shape为(2,2,**3**)。

↓
卷积核的(宽度W, 高度H, 通道C)



卷积神经网络 CNN

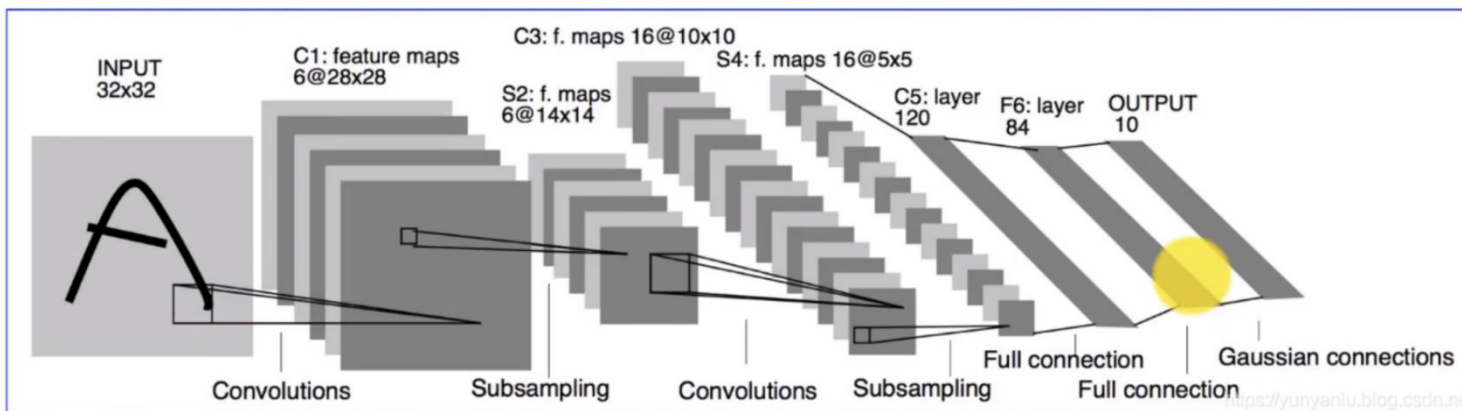
□ 架构设计 (Architecture Design for Neural Networks)



Yann LeCun



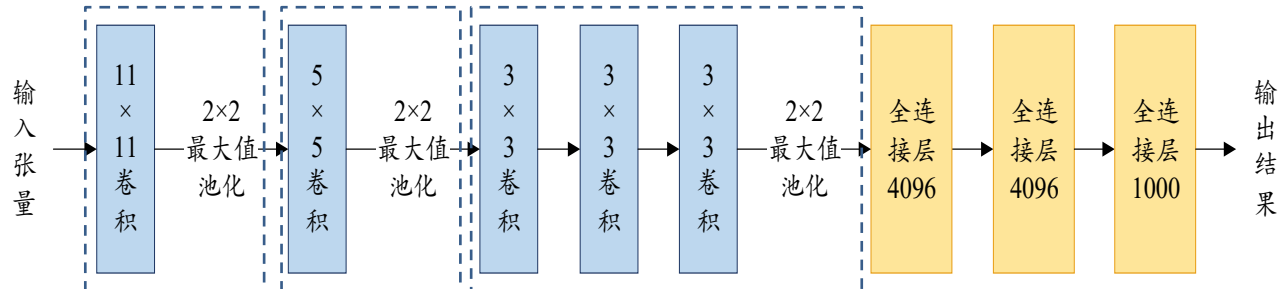
LeNet-5模型结构



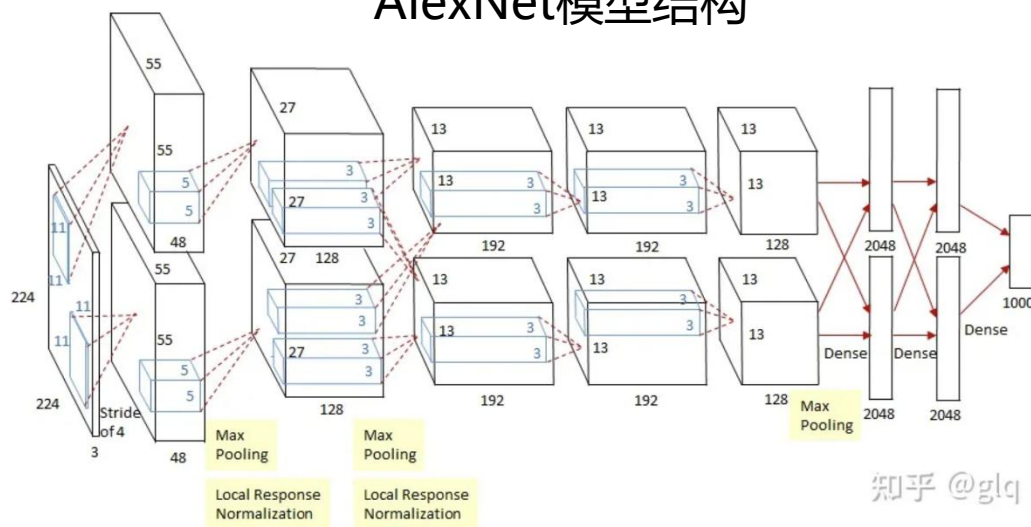
卷积神经网络 CNN

□ 架构设计

● 卷积-池化



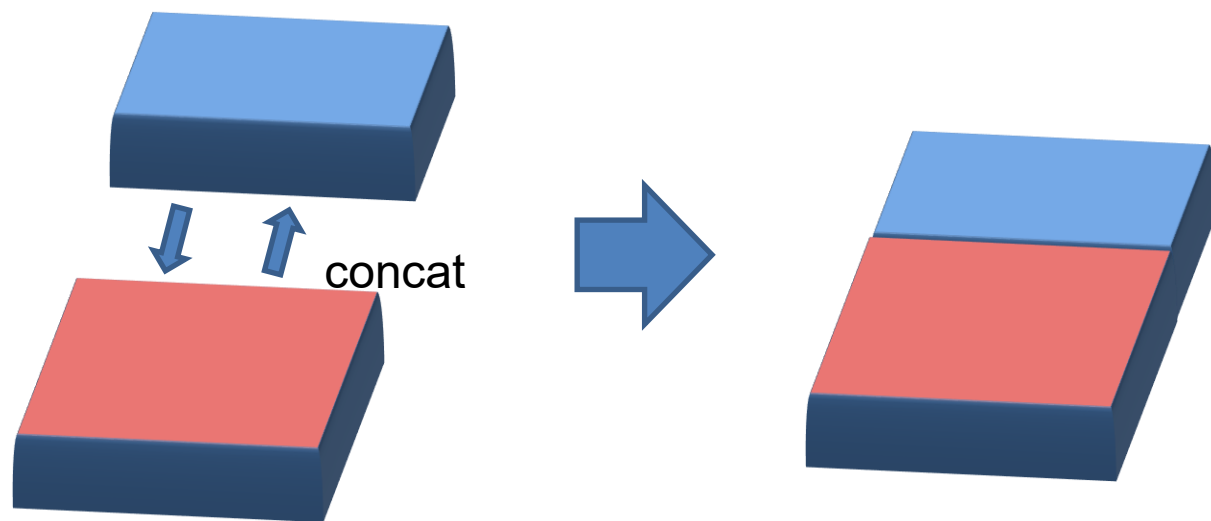
AlexNet模型结构



卷积神经网络 CNN

□ 架构设计

- concat: 将两个或者多个张量沿着某个维度进行拼接
- 举个例子: 一个 $(2 \times 3 \times 4)$ 的张量与一个 $(2 \times 6 \times 4)$ 的张量从第二维进行拼接, 可以得到一个 $(2 \times (3 + 6) \times 4)$ 的张量



由于concat操作可以自由地拼接张量, 通常能够达到“让模型同时考虑多种特征”的目的。需要注意, concat设计可以看作神经网络模型的超参数, 需要通过不断试验验证效果。

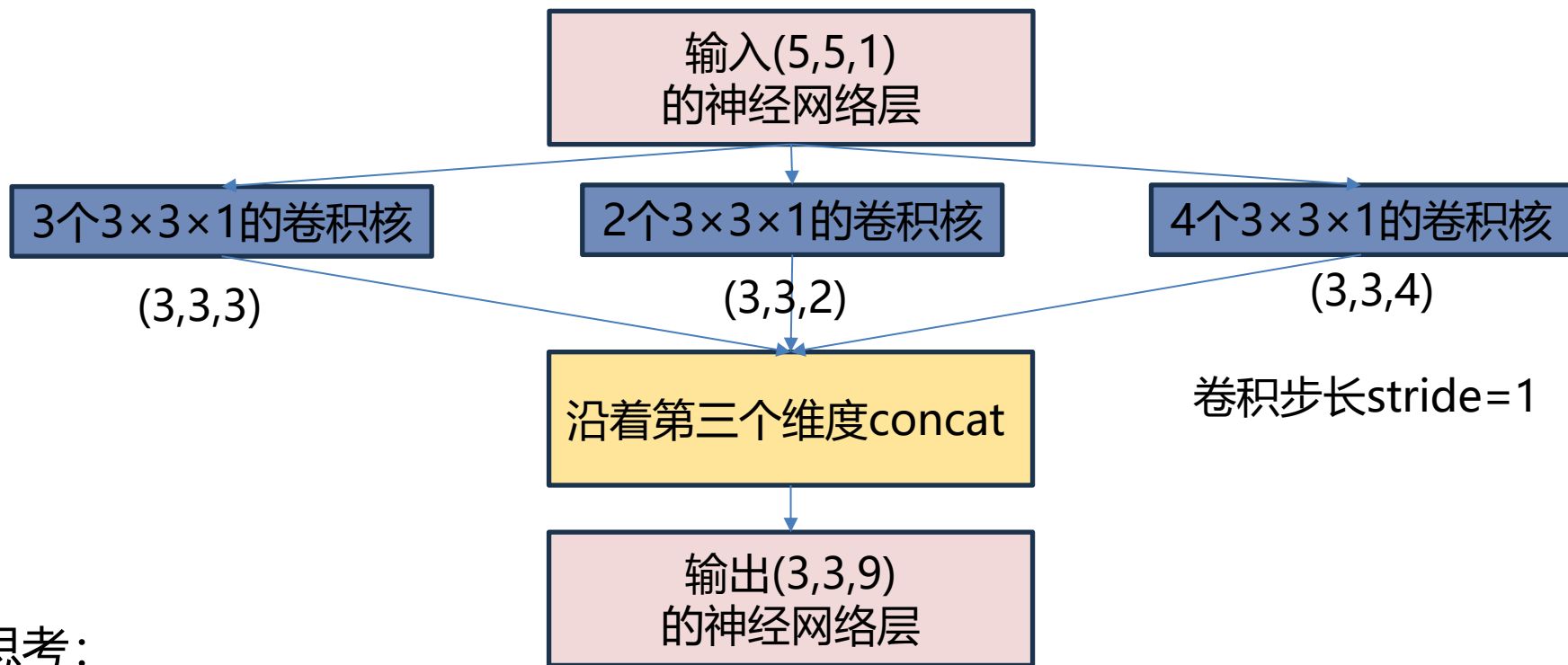


卷积神经网络 CNN

Inception Module: 是一个神经网络设计方式, 指在同一层内用多个不同的卷积核来提取不同特征, 在此基础上concat再输入下一层。

□ 架构设计

● 多卷积核 (Inception Module)



思考:

将不同卷积核输出的结果进行concat, 需要保证除拼接维度外的其它维度都必须相等, 如何做到?



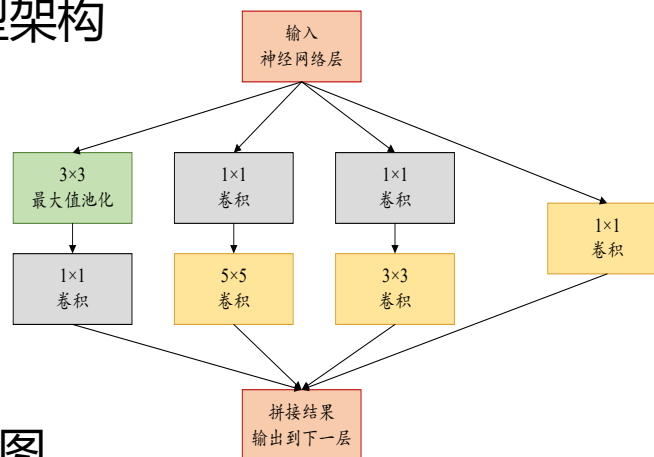
卷积神经网络 CNN

□ 架构设计

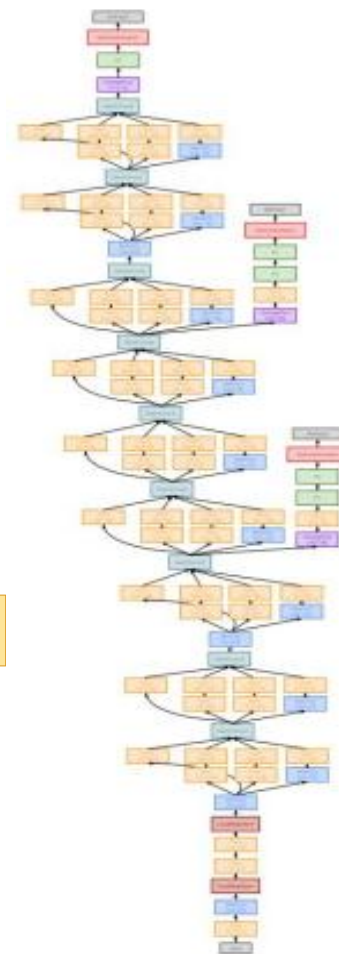
- 多卷积核

GoogLeNet提出了一种神经网络模型架构设计的全新思路：通过增加模型的“宽度”增加参数量，提高模型的学习能力。

右图展示了GoogLeNet网络结构示意图，GoogLeNet在每一层都用了多个卷积核提取不同特征(即Inception Module结构)。



Inception Module



GoogLeNet
示意图

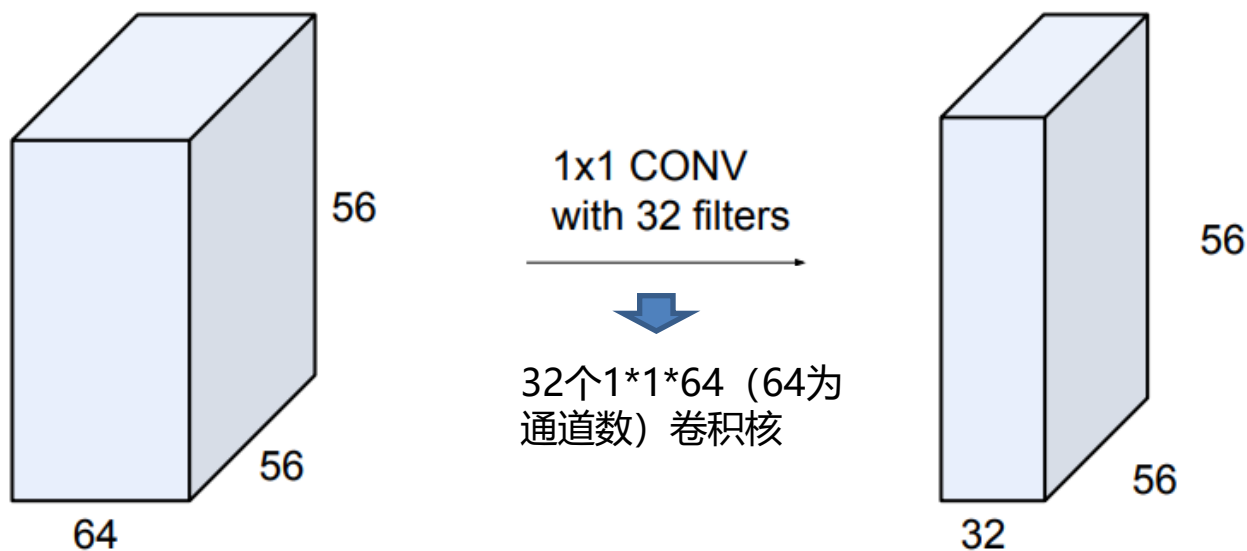


卷积神经网络 CNN

□ 架构设计

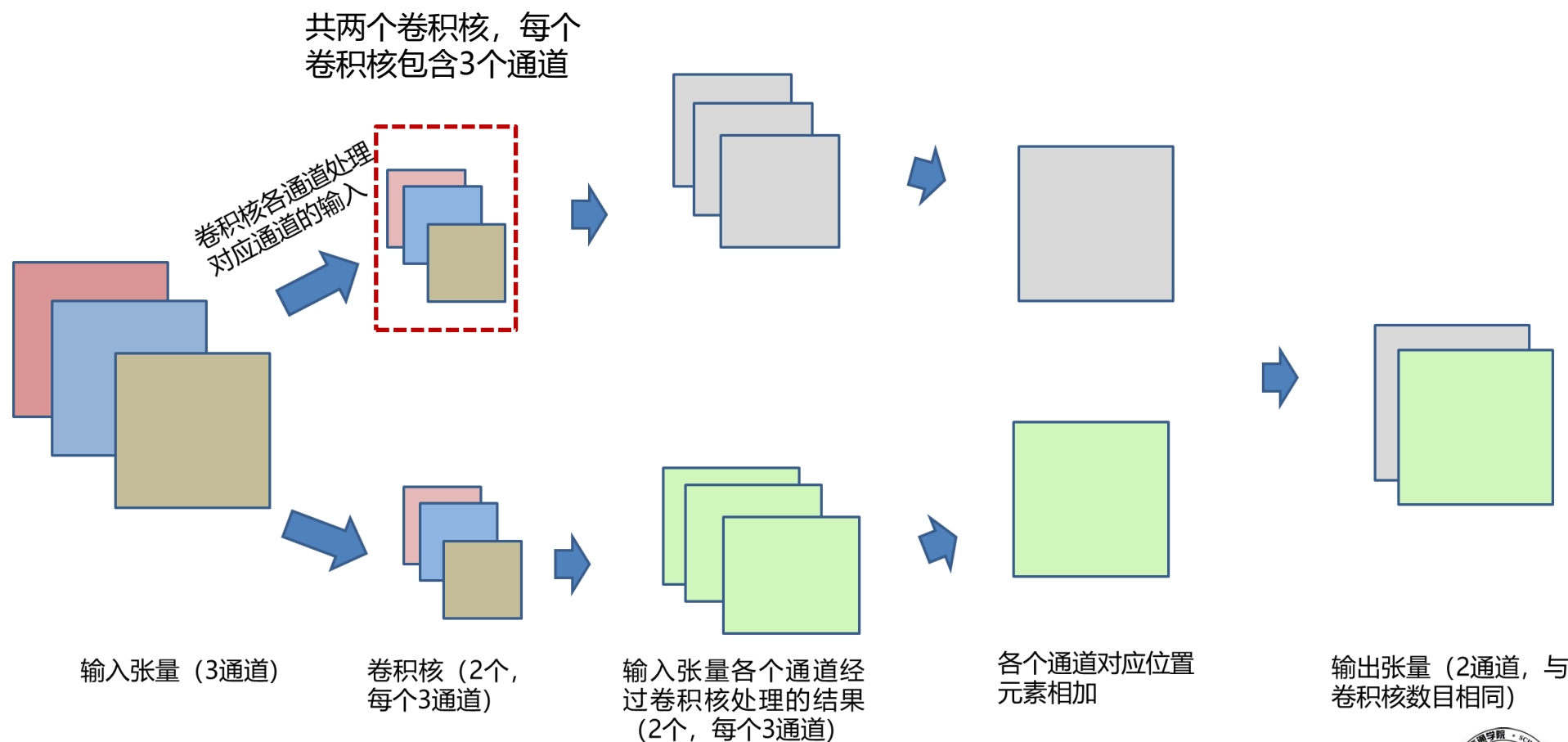
● 1*1卷积核

例题：如果输入张量的维度为 $56*56*64$ （其中64是通道数），卷积层包含32个 $1*1*64$ 维的卷积核（其中64是通道数），padding为0，stride为1。那么，输出张量的尺寸为？



多通道卷积

我们可以采用**多个多通道卷积核**对输入张量进行处理



输出张量的通道数量与卷积运算采用的卷积核数量相同

卷积神经网络 CNN

□ 架构设计

● 1*1卷积核

1*1卷积层的特点在于：使得输出能够保留了输入的shape，同时还能通过调整卷积核数量的方式调整通道数（通常是降低）。



“通过调整卷积核数量调整通道数” 是因为卷积运算输出张量的通道数一定与卷积核的数量相等，使用32个1*1*64的卷积核，输入张量的通道数可以从64降低到32

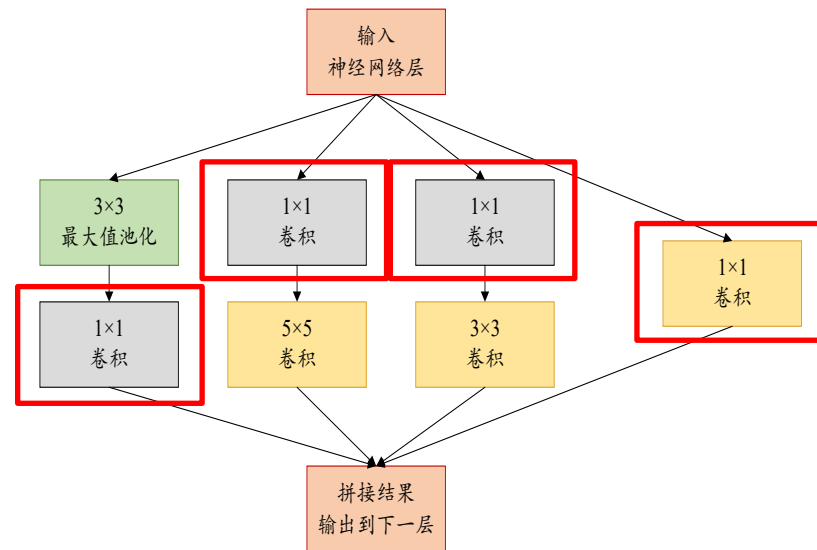
卷积神经网络 CNN

□ 架构设计

● 1*1卷积核

思考：Inception module中为什么
加入了很多1*1的卷积核？

“降低模型整体的计算量”

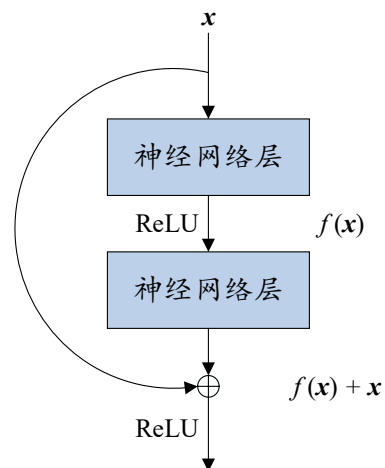


卷积神经网络 CNN

□ 架构设计

- 残差连接 (residual connection)

残差连接 (Residual Connection) 是指在神经网络中, 将前一层的输出直接与后一层的输入相加 (并不一定是相邻层), 从而构成一种跨层连接的方式。



$$x + f(x)$$

神经网络
某层的输
入张量

神经网络某层
的输出 (与 x 的
shape相同)

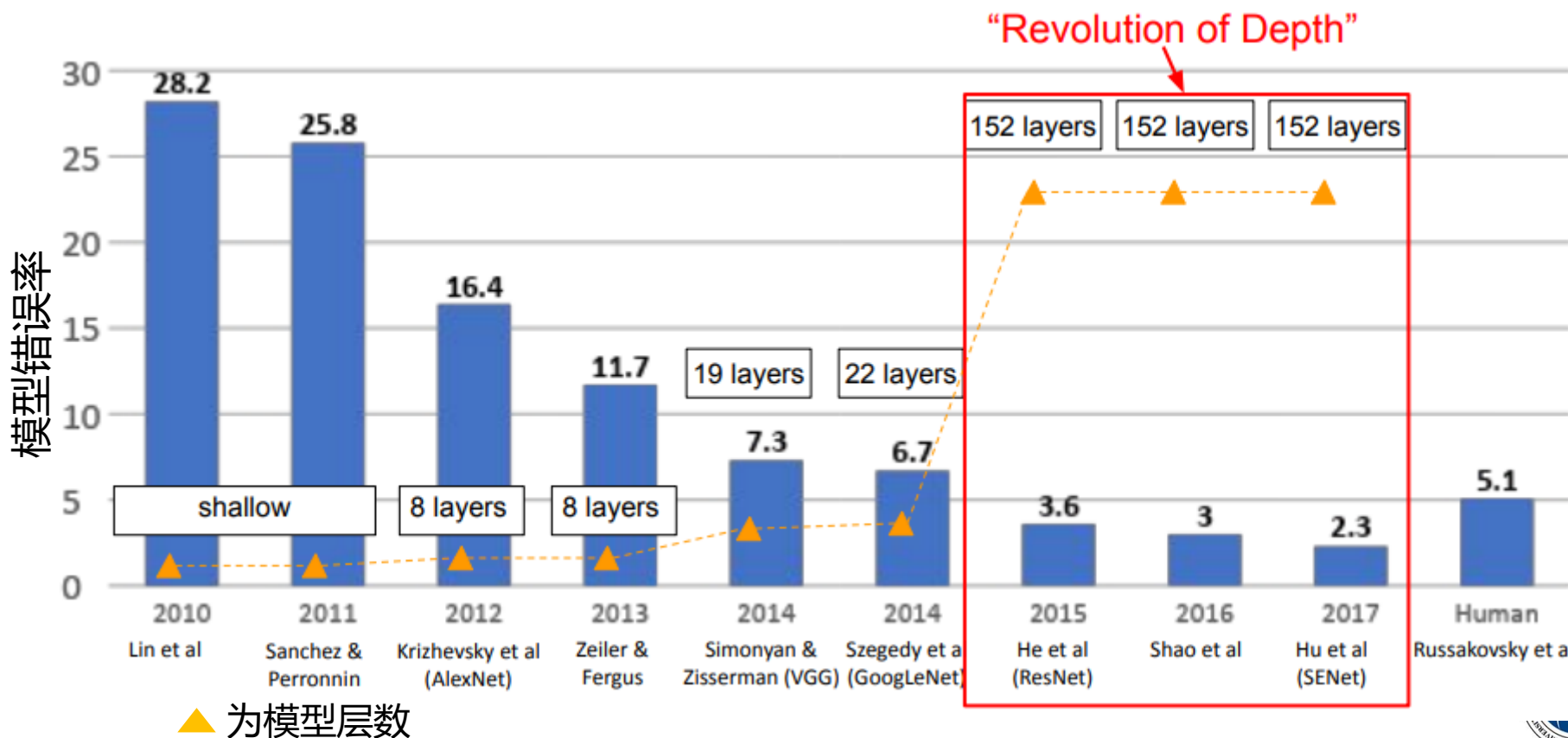
思考: 引入残差连接的深度学习模型的深度往往要深于普通的卷积神经网络模型, 但我们知道深度和精度并不是正相关, 那么残差连接的设计为什么会产生深度增加、精度也增加的现象?



卷积神经网络 CNN

□ 架构设计

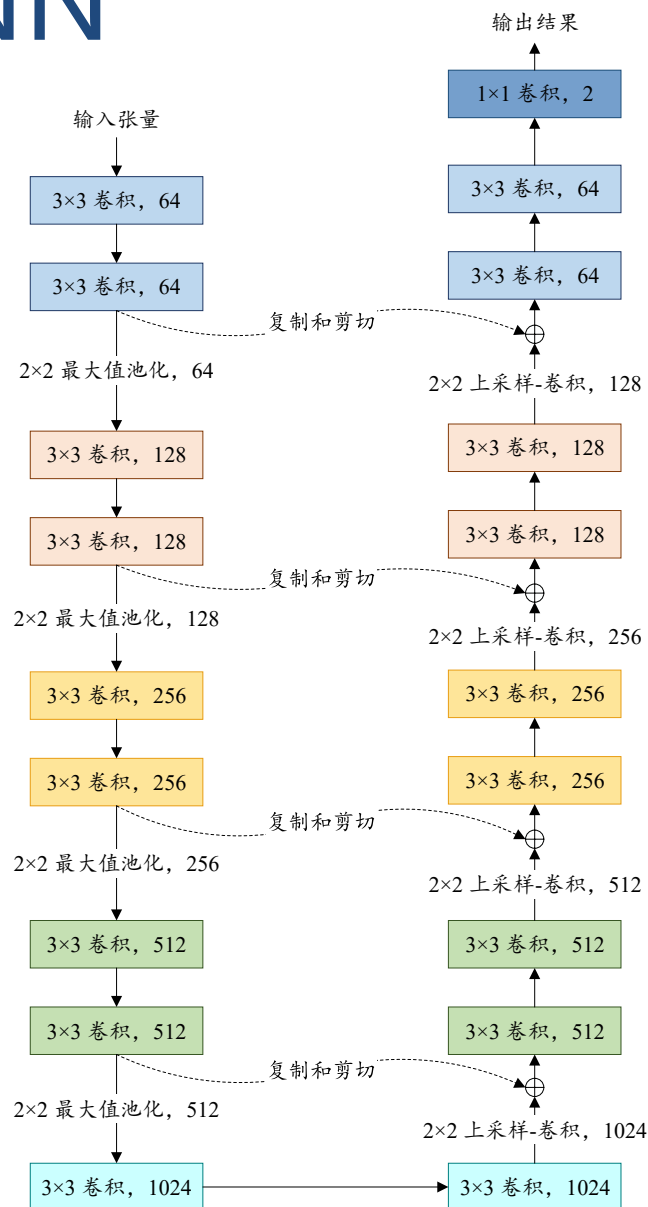
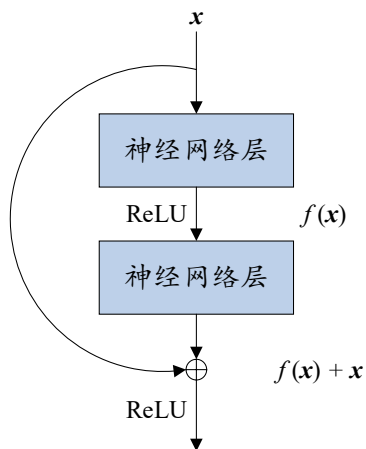
● 残差连接



卷积神经网络 CNN

□ 架构设计

● 残差连接



卷积神经网络 CNN

□网络性能

CNN通过以下模型设计方法提升模型性能

- 1、通过L2正则化、Dropout技术和扩展数据集等处理，有效缓解过拟合；
- 2、使用ReLU激活函数，该激活函数的导数为常量，可以缓解梯度消失/梯度爆炸等问题，加速模型训练；
- 3、在神经网络中设计卷积、池化和全连接层；
- 4、使用残差连接方式进行网络结构设计等。



卷积神经网络 CNN (阅读内容)

□卷积层的反向传播

反向传播时误差矩阵与原输入矩阵相乘得到梯度。

1、输入梯度计算

输入梯度需将卷积核旋转180度后与补零的误差矩阵做全卷积。

$$\frac{\partial J}{\partial X} = \delta \cdot K^T, \text{ 但需通过全卷积实现。}$$

2、权重梯度计算

同一卷积核在不同位置复用，反向传播时需累加所有位置局部梯度。

$$\frac{\partial J}{\partial K} = X^T \cdot \delta \quad (\delta \text{ 为后层传递的梯度})$$

□池化层的反向传播

1、最大池化

前向传播时记录最大值位置 (Max ID)，反向传播时梯度仅传回该位置，其他位置置零。

2、平均池化

反向传播时将梯度均分到前向池化

$$\nabla X_{pool\text{区域}} = \frac{\nabla Y}{k^2} \quad (k \text{ 为池化核大小})$$

3、无参数的纯信号传递

池化层无训练参数，反向传播仅是梯度上采样操作。



□ 例12-1. 基于CNN的网约车流量预测

● 背景分析与问题介绍：

网约车服务作为一种近年来涌现的新型出行方式，以其便捷、灵活和创新的运营策略受到出行者和交通管理人员关注。随着网约车市场规模的不断扩大，为了提升平台管理效率、合理制定调度策略、降低系统运营成本，如何准确地预测未来的流量成为亟待解决的问题。

本案例将基于**网约车历史轨迹数据**构造网约车流量预测模型。以网约车时空轨迹数据为输入，构造神经网络模型，**预测不同时空状态下网约车流量**。



数据集介绍:

本次案例所用**网约车历史轨迹数据**在原始车辆轨迹数据基础上，进行网格化和特征提取后的衍生数据，字段包含为：rowid（行id）、colid（列id）、time_id（时间网格id）、aveSpeed（网格平均速度）、gridAcc（网格平均加速度）、volume（网格浮动车流量）、speed_std（网格速度标准差）、stopNum（网格平均停车次数）、date（日期）。部分数据样例如下表所示

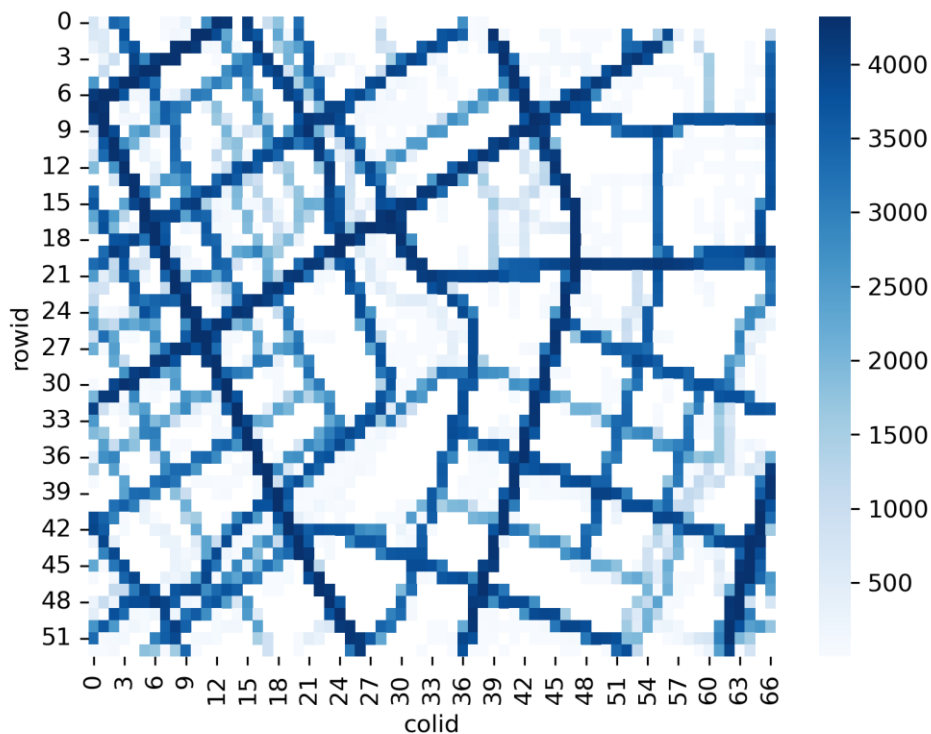
[illegible]

□ 例12-1. 基于CNN的网约车流量预测

● 应用场景解析：

1. 流量预测问题本质上属于**时间序列**的预测问题。
2. 网约车流量变化存在**时空属性**，模型必须充分考虑网约车流量在时间和空间两个维度的依赖关系。

- ① 本案例将研究区域被切分为 53×67 个矩形网格，此处我们将网格对应于图像中的像素点（每个像素值为流量值），使用卷积神经网络来捕捉**空间**流量信息之间的关联。
- ② 对于一系列**时间**维度上的流量分布，我们则将其对应于一系列堆叠的图像，作为图像的不同通道来进行处理。



□ 例12-1. 基于CNN的网约车流量预测

● 数据预处理：

获取图像每个像素点的值，即每个网格的集计流量值。

✓ 流量缺失值处理

- ① 网约车的**出行比例**不高，车辆轨迹无法覆盖全部区域
- ② 城市中大量用地为植被或建筑物等，**非城市道路**，网约车流量必然为0
- ③ 网格**划分粒度过细**
- ④ 网约车流量在**时间**上的**波动**变化造成的

寻找缺失值，**用0进行填补**，表示“网格流量为0”的事实。

●数据预处理：

获取图像每个像素点的值，即每个网格的集计流量值。

✓将顺序排列的历史流量记录整理成网格形式

问题1：网格划分粒度较细会导致**网格缺失数据**或**集计流量极低**（易受随机扰动影响，学习难度大）等问题。

问题2：对于网约车服务平台而，细粒度的时空网格**增加计算成本**，也不一定有**利于车辆的调度安排**。

本案例我们需对时空网格数据进行**聚合**。

1. 在空间维度上，本案例将 2×2 个原网格合并为新网格；
2. 在时间维度上，本案例选取**1小时**作为网格粒度。

将数据处理成标准输入格式，即样本 $\times$ 通道 $\times$ 宽 $\times$ 高。本案例中，对于待预测时刻 t ，我们选取了过去三小时($t - 3, t - 2, t - 1$)和一天前($t - 24$)的流量数据作为输入的4个通道。



● 模型构建：首先定义一个卷积模块，包含卷积层、归一化层、池化层

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Conv2D, BatchNormalization
from tensorflow.keras.layers import Activation, MaxPooling2D
from tensorflow.keras.regularizers import l2

# 卷积层定义
def conv_layer(inputs, num_filters=16, kernel_size=3, strides=1, data_format='channels_first',
               activation='relu', batch_normalization=True, maxpooling=True, pool_size=2,
               pool_strides=2):

    conv = Conv2D(num_filters, kernel_size=kernel_size, strides=strides, padding='same',
                  data_format=data_format, kernel_regularizer=l2(1e-4))
    x = conv(inputs)                                # 卷积
    if batch_normalization:
        x = BatchNormalization()(x)                 # 批归一化
    if activation is not None:
        x = Activation(activation)(x)               # 激活函数
    if maxpooling:
        x = MaxPooling2D(pool_size=pool_size, strides=pool_strides,
                          data_format=data_format, padding='same')(x)
    return x
```



- 模型构建：

由于卷积和池化的操作可能会使特征图的尺寸缩小，因此有必要定义一个上采样函数（池化逆操作），来对特征图尺寸进行还原。

```
from tensorflow.keras.layers import UpSampling2D

def upsample_layer(inputs, up_size=2,
                    interpolation='nearest',
                    data_format='channels_first'):

    # 上采样对象定义
    upsample = UpSampling2D(size=up_size,
                            data_format=data_format,
                            interpolation=interpolation)

    x = upsample(inputs)    # 上采样计算

    return x
```

逆池化参考：

Long J, Shelhamer E, Darrell T. Fully convolutional networks for semantic segmentation[C]//2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Boston, MA, USA: IEEE, 2015: 3431–3440.



- 模型构建：基于卷积层与上采样层建立完整的神经网络

```
from tensorflow.keras.layers import Input, Flatten, Dense
from tensorflow.keras.models import Model
```

```
def cnn_model(input_shape):
    inputs = Input(shape=input_shape)
    x = conv_layer(inputs, 16, pool_strides=1)
    x = conv_layer(x, 32, pool_strides=1)
    x = conv_layer(x, 32, pool_strides=1)
    x = conv_layer(x, 16)
    x = upsample_layer(x, 2)
    x = conv_layer(x, 1, maxpooling=False)
    # 将conv计算输出的张量展平，通过二层神经网络挖掘信息
    y = Flatten(data_format='channels_first')(x)
    y = Dense(128, activation='relu')(y)
    y = Dense(128, activation='relu')(y)
    outputs = Dense(100, activation='relu')(y)
    # 建立模型
    model = Model(inputs=inputs, outputs=outputs)
    return model
```



□ 例12-1. 基于CNN的网约车流量预测

● 模型训练及性能评估:

数据集划分: 案例选取数据集DATASET-B中2016年11月1日至2016年11月29日的数据作为**训练集**, 其中20%的数据作为**验证集**, 将11月30日的数据作为测试集。

模型训练细节: 模型使用Adam优化器进行优化, 学习率设置为0.0004, 并以MSE作为目标函数。在训练过程中, 32个样本为一个训练批次, 共训练50轮。值得注意的是, 在训练过程中, 执行**早停策略**, 避免模型产生严重的过拟合。

早停策略:

当模型在验证集上的表现在连续多次 (设定阈值) 验证过程中都无法提高, 则提前停止训练过程。

验证过程: 模型更新n个迭代轮次之后进行一次验证 用于判断有没有在验证集上过拟合。n是自行设置的超参数



□ 例12-1. 基于CNN的网约车流量预测

● 模型训练及性能评估：

“在训练过程中，32个样本为一个**训练批次**” 其中，训练批次是训练数据的一个子集，表示更新一次模型参数所用的数据

例如，总共800条数据的数据集可以划分成16个批次，每个批次50条数据。在模型训练中，第一轮训练用第一个批次（50条）的数据，基于梯度调整一次参数之后，用第二个批次的数据（50条）进行第二轮训练，直到所有批次数据训练完）

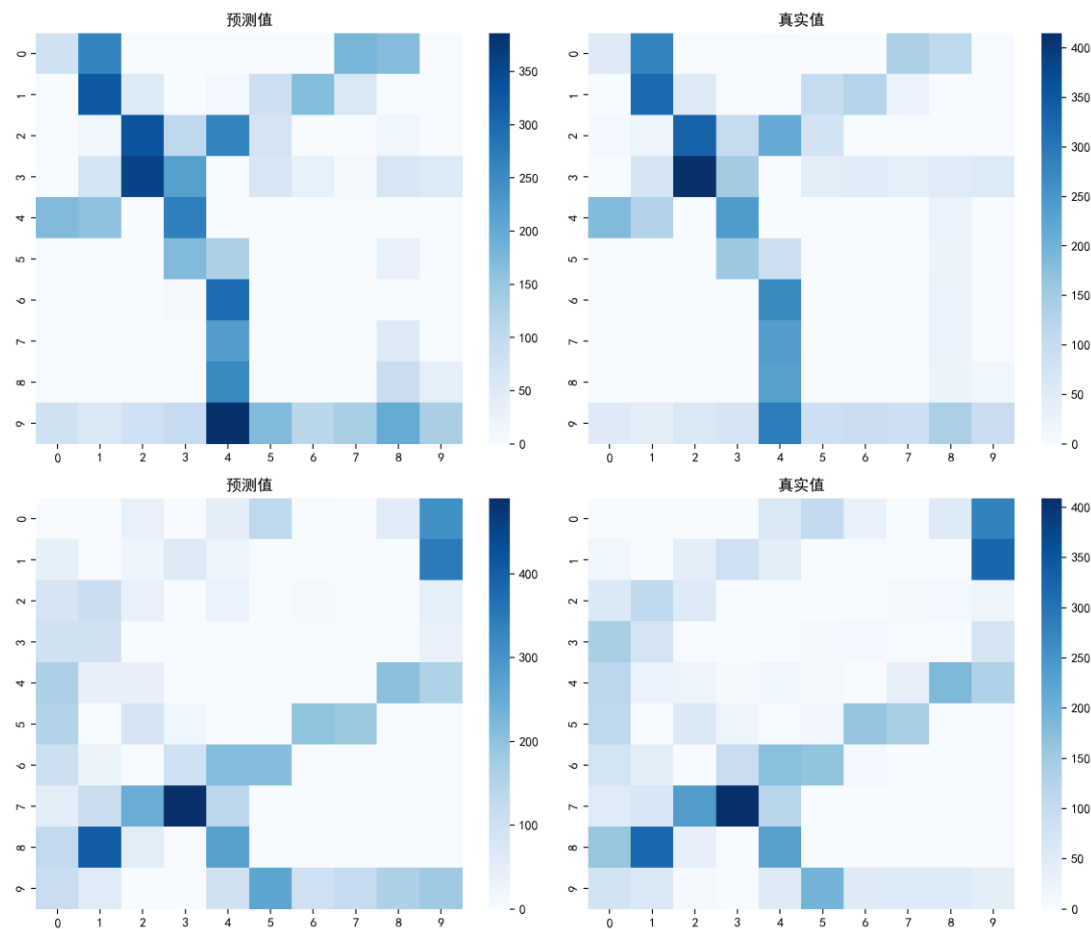


例12-1. 基于CNN的网约车流量预测

● 模型训练及性能评估:

MAE	MSE	RMSE
19.876	1431.725	37.838

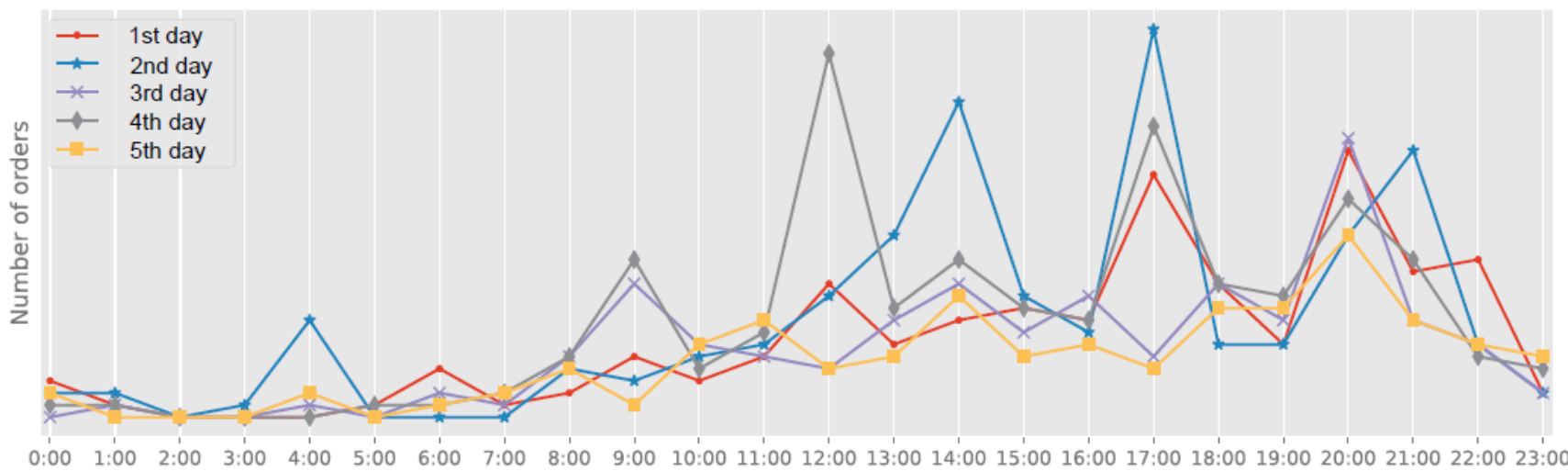
$$RMSE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}}$$



循环神经网络 RNN

□建模动机

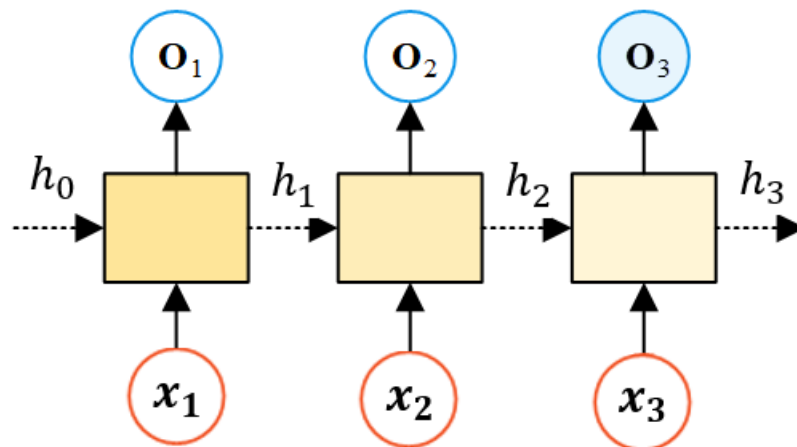
- 交通领域中最普遍的序列数据是**时间序列数据**，其往往具有显著的前后依赖关系，即未来某一时刻的输出结果与过去的状态紧密相关
 - 相关性：以某个路段的**流量/速度时间序列数据**为例，拥堵的传播与消散发生时，都会对后续时段产生影响，使得某一时段的交通量与过去了几个时段严格相关；
 - 规律性：以**出租车需求**为例：不同工作日同一时间段的需求量**相似**。



循环神经网络 RNN

□基本结构

- 循环神经网络在结构上引入了不同输入时间维度上的循环连接，在不同的时间点之间添加了新的隐藏层，使整个神经网络拥有了对序列前后关系进行刻画的能力。循环神经网络**某个时刻的输出信息**，除了作为运算结果输出，也可以成为**下一时刻的输入**，影响下一时间点的输出结果。



循环神经网络基本结构

循环神经网络 RNN

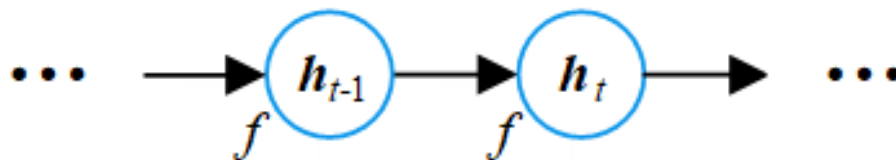
□基本结构

首先考虑一个长度为 N 的序列 $\{h_1, h_2, h_3, \dots, h_N\}$ 。下面建立一个**简单的模型**刻画时序数据的**前后依赖关系**，其中 t 时刻的序列状态 h_t 仅仅依赖前一时刻的序列状态 h_{t-1} ，这种关系可以写作：

$$h_t = f(h_{t-1})$$

为 $f(*)$ 代表不同时刻状态之间的**依赖映射**，为该映射添加**可调整（可学习）参数** θ_t ，这种关系可以写为：

$$h_t = f(h_{t-1}; \theta_t)$$



循环神经网络 RNN

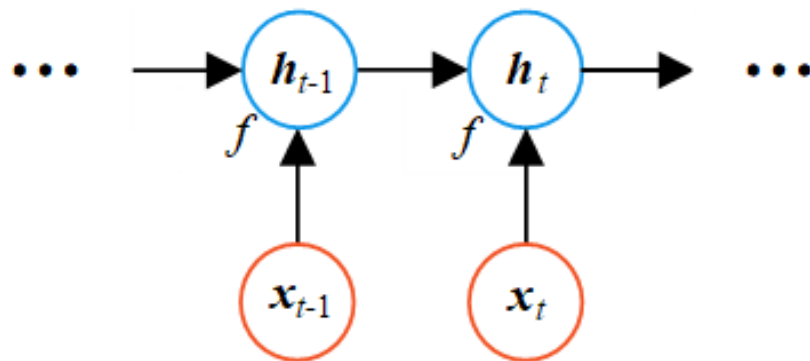
□基本结构

真实场景下的序列通常不是一个孤立系统，会受**诸多外部要素影响**

比如交通需求（路段流量、OD等）通常还受到空间因素（地理空间上临近的预测对象的需求情况）、外部因素影响（天气、能见度、特殊事件）等等。

在简单循环序列的基础上，我们可以继续添加更多的外部变量 x_t 作为输入，来刻画各类外部要素。

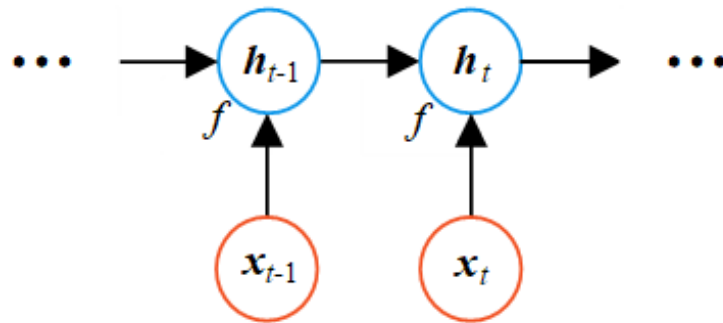
$$h_t = f(h_{t-1}, x_t; \theta_t)$$



循环神经网络 RNN

□ 基本结构

$$h_t = f(h_{t-1}, x_t; \theta_t)$$



h_t 是一个定长的向量，可以视为对过去“**所有时间点**”输入数据的一种压缩表示。其中，初始输入 h_0 通常被设为零向量。因此，时序状态的依赖关系也可以展开为关于 t 的函数 $g_t(*)$ ：

$$\begin{aligned} h_t &= f(h_{t-1}, x_t; \theta_t) = f(f(h_{t-2}, x_{t-1}; \theta_{t-1}), x_t; \theta_t) = \dots \\ &= g_t(x_1, x_2, \dots, x_t; \theta_1, \theta_2, \dots, \theta_t) \end{aligned}$$

由此可见， h_t 的这种压缩表示能够方便处理任意长度的序列，我们不再需要对每一个时刻 t 单独去学习一个新的映射 g_t ，而可以用一个统一的映射 f 来表示不同时刻的依赖关系。



循环神经网络 RNN

□基本结构

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\theta}_t)$$

类似于CNN，RNN在设计时为了减小模型计算成本，采用了**参数共享**的机制，如下式所示：

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\theta}) = f(f(\mathbf{h}_{t-2}, \mathbf{x}_{t-1}; \boldsymbol{\theta}), \mathbf{x}_t; \boldsymbol{\theta}) = \dots = g_t(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t; \boldsymbol{\theta})$$

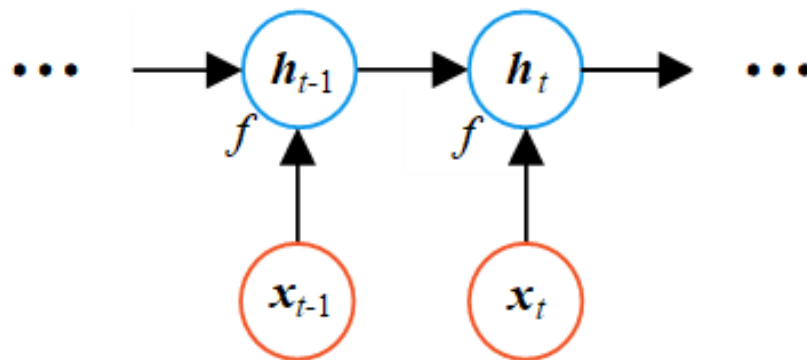
即每个时刻共享同一个依赖映射 f 和可学习参数 θ ，所有的RNN神经元均共享同样的可学习参数 θ 进行前向传播与反向传播；这一设计在简化了模型的同时，提升了模型的训练速度。



循环神经网络 RNN

□ 基本结构

$$h_t = f(h_{t-1}, x_t; \theta)$$



假设依赖映射函数为 $f(h_{t-1}, x_t; \theta) = ah_{t-1} + bx_t$

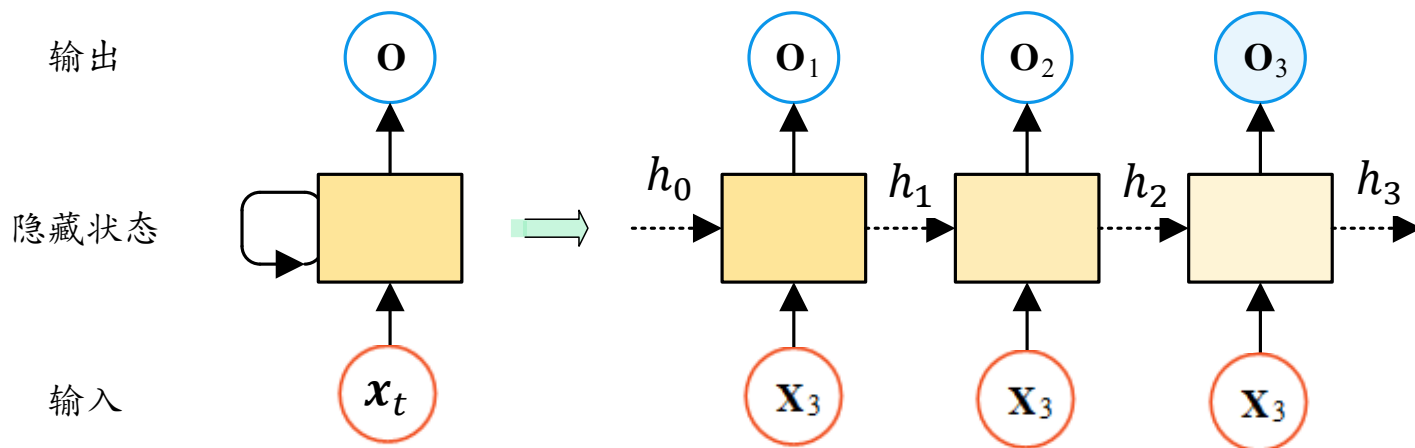
可见，该式中 θ 只包含两个可学习参数 a 与 b 。

在RNN的**参数共享**机制作用下，此时 $g_t(x_1, x_2, \dots, x_t; \theta)$ 可以表示为：

$$g_t(x_1, x_2, \dots, x_t; \theta) = bx_t + abx_{t-1} + \dots + a^{t-1}bx_1$$

循环神经网络 RNN

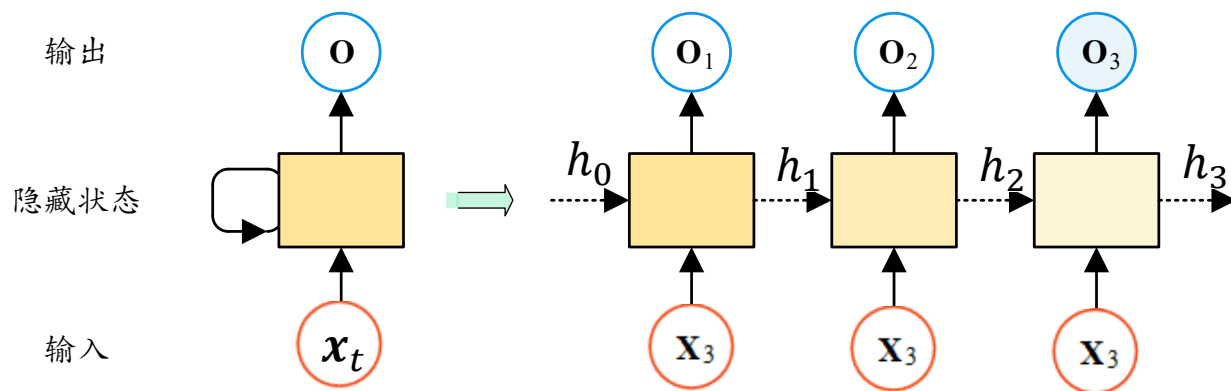
□基本结构：为每一个时刻 t 再添加一个输出 O ，这样我们就建立了一个完整的循环神经网络（左侧是对右侧计算图的简化形式）：



在多维度的情况下，每个循环神经元内部的运算应转化为**矩阵运算**。因此上一页ppt中参数 a 调整为**隐藏状态 h_t 的权重矩阵 W_{hh}** ，参数 b 调整为输入变量 x_t 的权重矩阵 W_{hx} ，并新增偏置项 b_h

循环神经网络 RNN

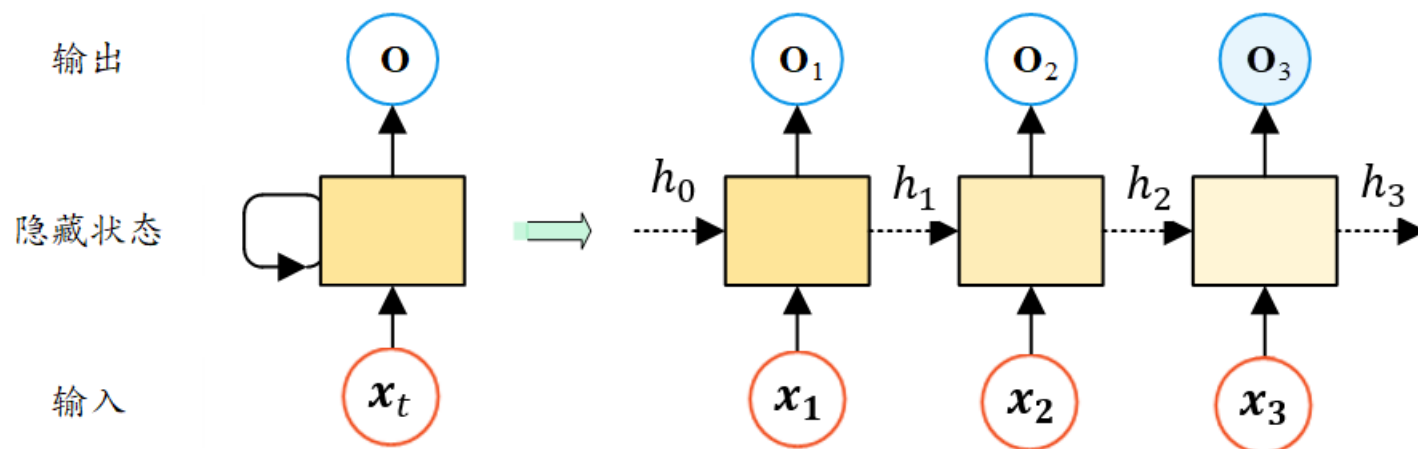
□交通中 x , h , o 的形象解释:



- 1、输入 x ——每个时间步（如每10分钟）一个网格内的网约车流量
- 2、隐藏状态 h ——通过隐藏状态积累历史交通规律（如时序信息的累积，隐藏状态 $h(t-1)$ 编码了过去24小时的流量波动）捕捉网约车流量变化规律：
 - (1) 周期性规律：学习网格的流量高峰模式，如工作日早高峰通勤流量增加；
 - (2) 外部因素响应：通过LSTM动态调整权重，如雨天时增强天气特征的记忆。
- 3、输出 o ——下一时间步的网格内网约车流量预测值，若需预测未来多个时间步（如未来30分钟），可通过迭代输出（将当前预测作为下一时间步的输入）实现。

循环神经网络 RNN

□基本结构



$$h_t = \tanh(W_{hx}x_t + \boxed{W_{hh}h_{t-1}} + b_h)$$

与前馈神经网络的主要差异

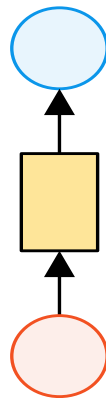
$$o_t = W_{oh}h_t + b_o$$

循环神经网络 RNN

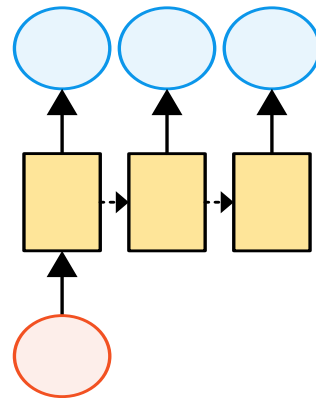
□基本结构

一般的前馈神经网络只能建立输入到输出的一对一映射，但是循环神经网络得益于其内部的时间步概念，可以拓展到序列间的映射。循环神经网络能够建立一对多、多对一或者多对多的映射关系。

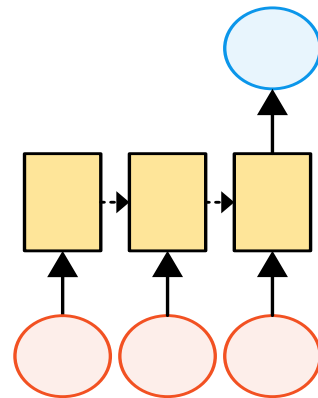
一对一



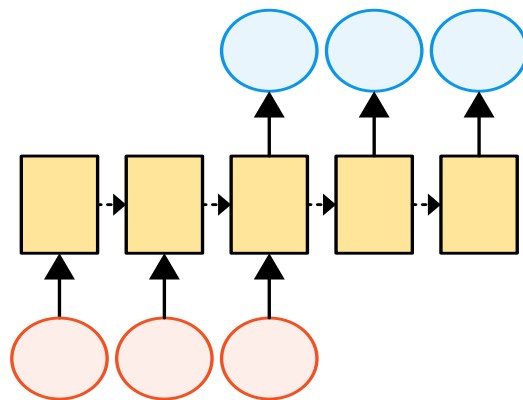
一对多



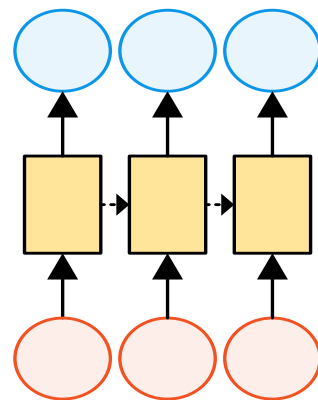
多对一



多对多



多对多



循环神经网络 RNN

□ 基本结构

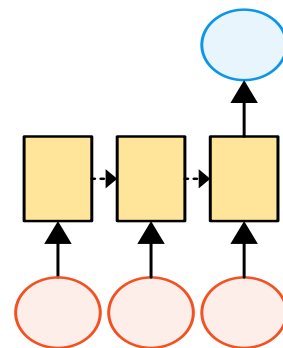
在交通预测任务中（包括数据驱动的微观交通流建模、数据驱动的宏观/路网需求预测等等），一般“多对一”与“多对多”的形式映射较为常见：

✓ 多对一

单步预测任务：基于历史的状态，预测未来一个时间步的状态

输入：多个历史时间步的状态

输出：待预测（一个）时间步的状态



循环神经网络 RNN

□ 基本结构

在交通预测任务中（包括数据驱动的微观交通流建模、数据驱动的宏观/路网需求预测等等），一般“多对一”与“多对多”的形式映射较为常见：

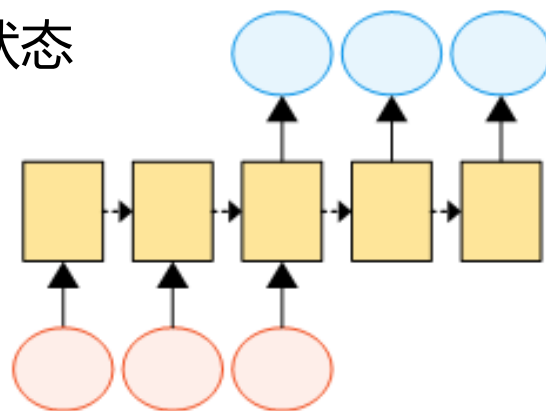
✓ 多对多

多步预测任务：基于历史交通状态，预测未来

连续多个时间步的交通状态

输入：多个历史时间步的状态

输出：待预测多个时间步的状态



循环神经网络 RNN

□按时间步 (time step) 反向传播

参见教材12.2.2节的推导可得以下公式 (T为时间步的总长度)：

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hx}} &= \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hx}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \mathbf{x}_t^\top \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} &= \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \mathbf{h}_{t-1}^\top\end{aligned}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \left[\sum_{i=t}^T (\mathbf{W}_{hh}^\top)^{T-i} \mathbf{W}_{oh}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{o}_{T+t-i}} \right]$$

思考：上面公式中 $\sum_{i=t}^T$ 的含义是？

[t,T]各个时间步的梯度之和

如何理解？

见下文所示。

循环神经网络 RNN

按时间步反向传播

思考：

$\sum_{i=t}^T$ 的含义是 $[t, T]$ 各个时间步的梯度之和。如何理解？

右图中蓝色的框是RNN的输出 $\mathbf{o}_t$ ，红色的框是每个时间步的外部输入 $\mathbf{x}_t$ ，绿色的框是隐藏状态 $\mathbf{h}_t$ 。

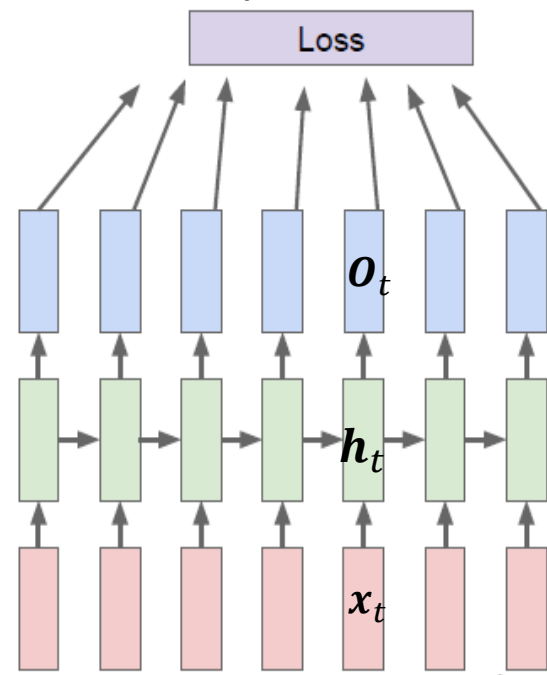
以交叉熵损失函数为例，RNN在t时刻的损失函数写成如下形式：

$$\text{Loss}_t = -[\mathbf{y}_t \ln(\mathbf{o}_t) + (\mathbf{y}_t - 1) \ln(1 - \mathbf{o}_t)]$$

其中， $\mathbf{y}_t$ 是真实的标签值

全部T个时刻的损失函数表达为如下形式：

$$\text{Loss} = \sum_{t=1}^T \text{Loss}_t = - \sum_{t=1}^T [\mathbf{y}_t \ln(\mathbf{o}_t) + (\mathbf{y}_t - 1) \ln(1 - \mathbf{o}_t)]$$



循环神经网络 RNN

□按时间步反向传播

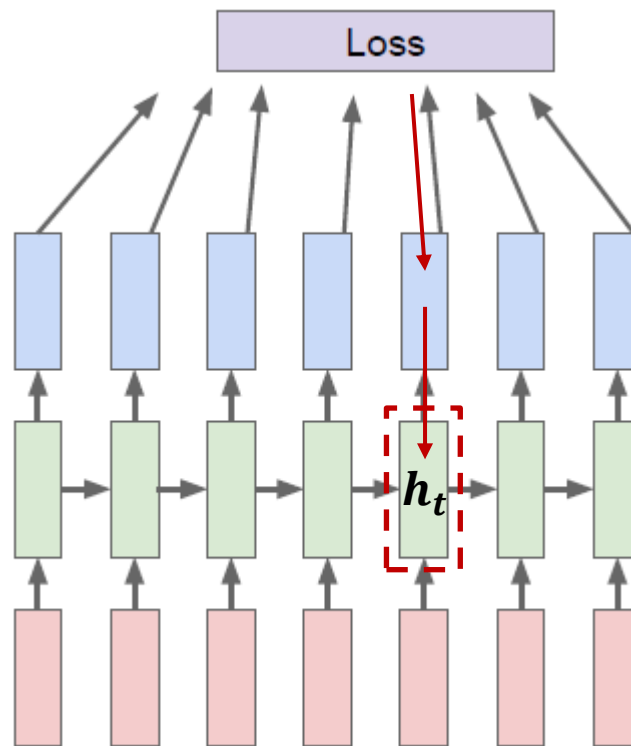
思考：

$\sum_{i=t}^T$ 的含义是 $[t, T]$ 各个时间步的梯度之和。如何理解？

红色虚框对应时间步权重的梯度分析

梯度来源1：

时间步 t 的输出反向传播至 h_t



循环神经网络 RNN

按时间步反向传播

思考：

$\sum_{i=t}^T$ 的含义是 $[t, T]$ 各个时间步的梯度之和。如何理解？

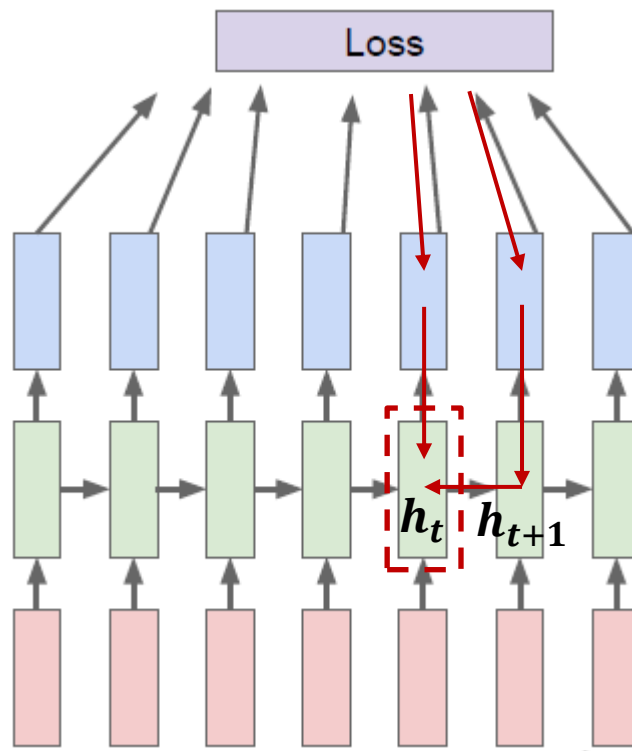
红色虚框对应时间步权重的梯度分析

梯度来源1：

时间步 t 的输出反向传播至 h_t

梯度来源2：

时间步 $t+1$ 的输出反向传播至 h_t



循环神经网络 RNN

按时间步反向传播

思考：

$\sum_{i=t}^T$ 的含义是 $[t, T]$ 各个时间步的梯度之和。如何理解？

红色虚框对应时间步权重的梯度分析

梯度来源1：

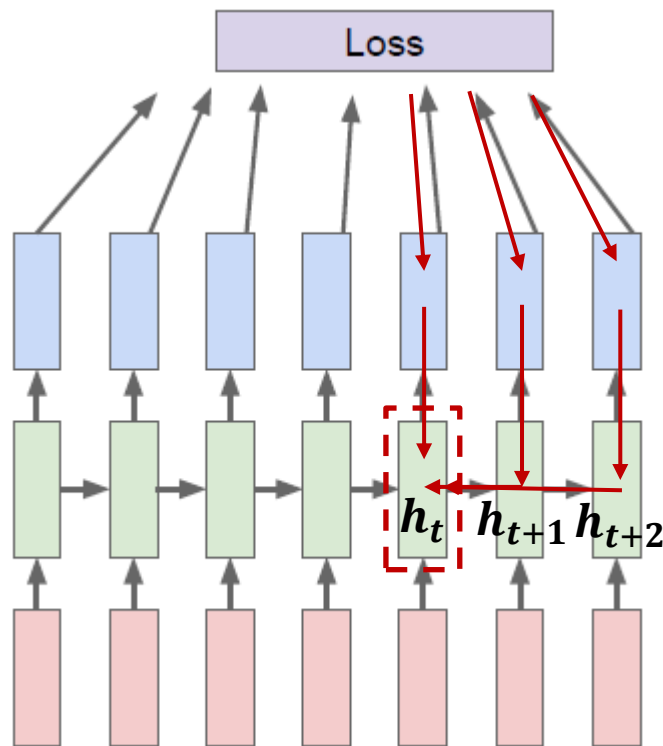
时间步 t 的输出反向传播至 h_t

梯度来源2：

时间步 $t+1$ 的输出反向传播至 h_t

梯度来源3：

时间步 $t+2$ 的输出反向传播至 h_t



循环神经网络 RNN

□按时间步反向传播

根据教材12.2.2节的推导可得以下公式：

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hx}} &= \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hx}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \mathbf{x}_t^\top \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} &= \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \mathbf{h}_{t-1}^\top\end{aligned}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \sum_{i=t}^T (\mathbf{W}_{hh}^\top)^{T-i} \mathbf{W}_{oh}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{o}_{T+i-i}}$$

- 上面公式中存在指数项，这使得较长的时间步传递的梯度很可能会变得极大或极小，也就是**梯度爆炸**和**梯度消失**现象，对模型训练造成严重影响。

$$|\mathbf{W}_{hh}| > 1 \quad |\mathbf{W}_{hh}| < 1$$

- 梯度爆炸：导致模型难以正常收敛，模型的损失函数值与权重更新剧烈波动
- 梯度消失：模型权重的更新会变得十分缓慢，以至于无法从样本中进行有效的学习，损失函数值几乎停止继续下降。



循环神经网络 RNN

□按时间步反向传播

根据教材12.2.2节的推导可得以下公式：

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hx}} &= \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hx}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \mathbf{x}_t^\top \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} &= \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \mathbf{h}_{t-1}^\top\end{aligned}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \sum_{i=t}^T (\mathbf{W}_{hh}^\top)^{T-i} \mathbf{W}_{oh}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{o}_{T+i-i}}$$

- **梯度爆炸和梯度消失**现象，会对模型训练造成严重影响。
- 应对方法是：对梯度进行限制，即梯度裁剪（gradient clipping）。当参数的梯度大于某个阈值时，强制将其等于该阈值。但梯度裁剪的方法无法解决梯度消失问题。

循环神经网络 RNN

□按时间步反向传播

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hx}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hx}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \mathbf{x}_t^\top$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \mathbf{h}_{t-1}^\top$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \sum_{i=t}^T \underbrace{(\mathbf{W}_{hh}^\top)^{T-i} \mathbf{W}_{oh}^\top}_{\text{red dashed box}} \frac{\partial \mathcal{L}}{\partial \mathbf{o}_{T+t-i}}$$

思考：

何为梯度消失？一定是梯度为0吗？

循环神经网络 RNN

按时间步反向传播

思考：

何为梯度消失？一定是梯度为0吗？

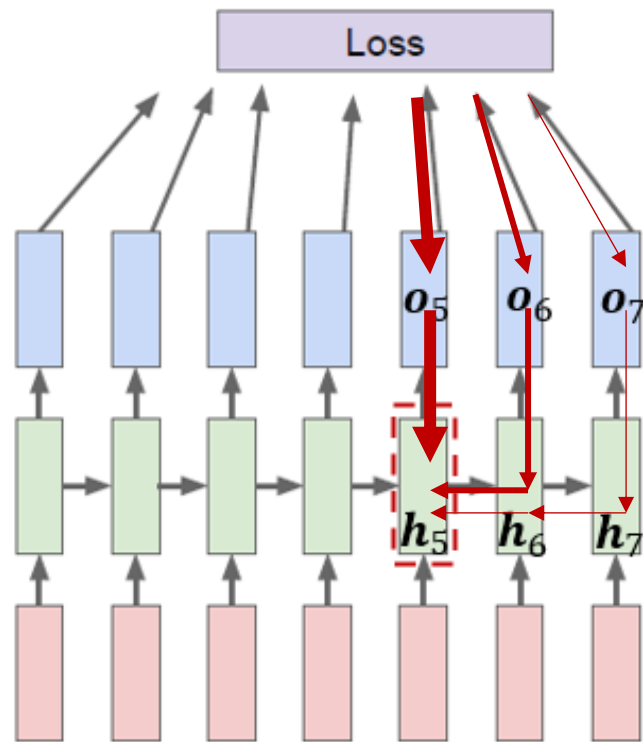
$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \sum_{i=t}^T (\mathbf{W}_{hh}^T)^{T-i} \mathbf{W}_{oh}^T \frac{\partial \mathcal{L}}{\partial \mathbf{o}_{T+i-i}}$$

No!

如果箭头粗细代表梯度大小，
即使 $W_{hh} < 1$ ，右图最大的梯度路径也不会
为0，只是取值很小而已。

如何正确理解“梯度消失”呢？

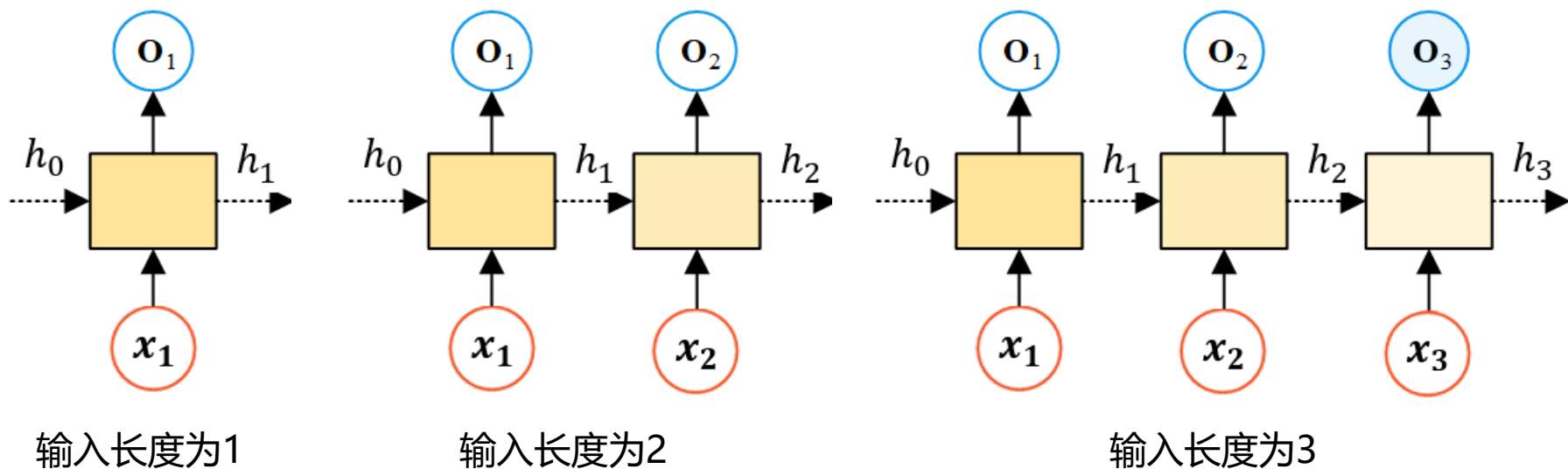
$$\begin{aligned} \frac{dL}{dh_5} &= (W_{hh}^T)^2 W_{oh}^T \frac{dL}{do_7} + (W_{hh}^T)^1 W_{oh}^T \frac{dL}{do_6} \\ &\quad + (W_{hh}^T)^0 W_{oh}^T \frac{dL}{do_5} \end{aligned}$$



循环神经网络 RNN

□ RNN神经网络的优点：

1. 可以处理任何长度的输入（只需要增加神经元数量）



思考：

输入长度增加后，模型参数量会增加吗？计算量呢？

循环神经网络 RNN

□ RNN神经网络的优点：

2. 理论上（模型设计上），在计算第 t 步时，能够使用之前所有时间步的信息。

□ RNN神经网络的缺点：

1. 很难使用很多时间步之前的信息，会产生“遗忘”；
2. 经典RNN结构容易出现“梯度爆炸”与“梯度消失”，使得BP算法无法有效地进行参数更新，模型难以训练、收敛；
3. 前向传播计算慢，必须按照时间步依次计算，无法并行。



循环神经网络 RNN

□ 发展历程

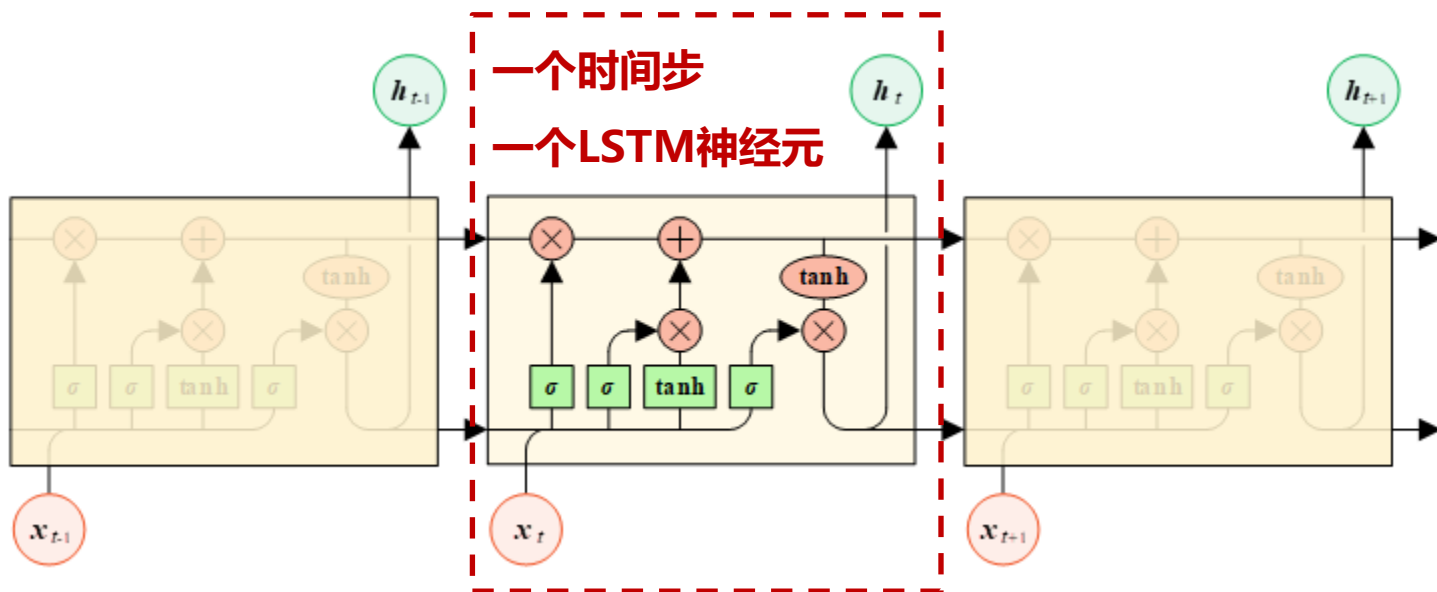
- 1982年，John Hopfield发明了最早的RNN的雏形Hopfield Network，解决组合优化问题。
- 1986年，Jordan定义了Recurrent的概念，提出Jordan Network。1990年，Elman对该网络进行了简化，并采用BP算法进行训练，经典RNN模型诞生。但由于RNN梯度消失及梯度爆炸的问题，训练非常困难，应用非常受限。
- 1997年，Schmidhuber提出的长短期记忆模型(LSTM)使用门控单元及记忆机制大大缓解了早期RNN训练的问题。



循环神经网络 RNN

□长短期记忆模型 (Long Short Term Memory, LSTM)

- **门控结构** (gated unit)。由于梯度爆炸和梯度消失，RNN很难处理长时间步之间的关联，因此门控结构应运而生。借助这些门，可以对信息在时间维度上的流动加以控制，让神经网络自行控制信息的传递。
- LSTM是最经典的带有门控结构的循环神经网络



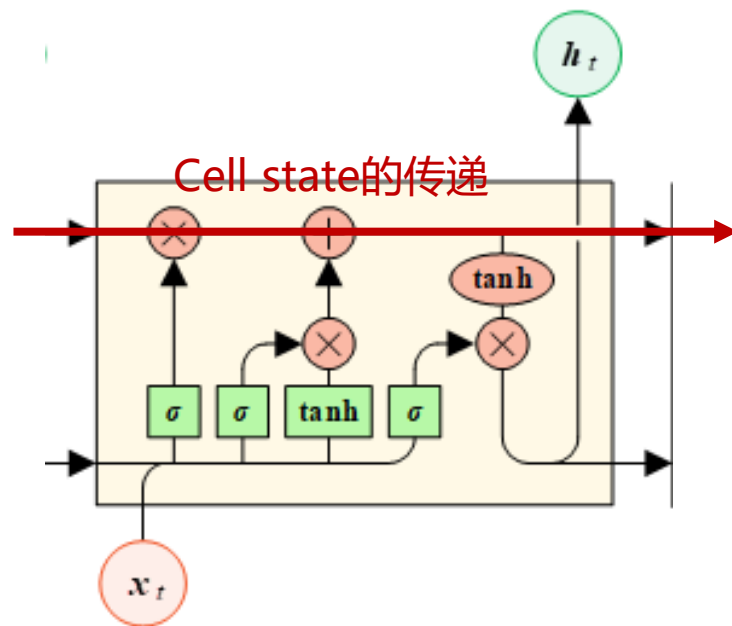
循环神经网络 RNN

□长短期记忆模型 (LSTM)

- 细胞状态 (cell state) 存储之前时间步的信息。
- LSTM神经元中含有三个门控单元：
 - 遗忘门 (forget gate)
 - 输入门 (input gate)
 - 输出门 (output gate)

这些门负责细胞状态中信息的添加和移除，借助于激活函数控制信息的流动。

- 相较于经典RNN，LSTM使用四个特殊的神经网络层（图中四个绿色的激活函数）取代了单个全连接层。

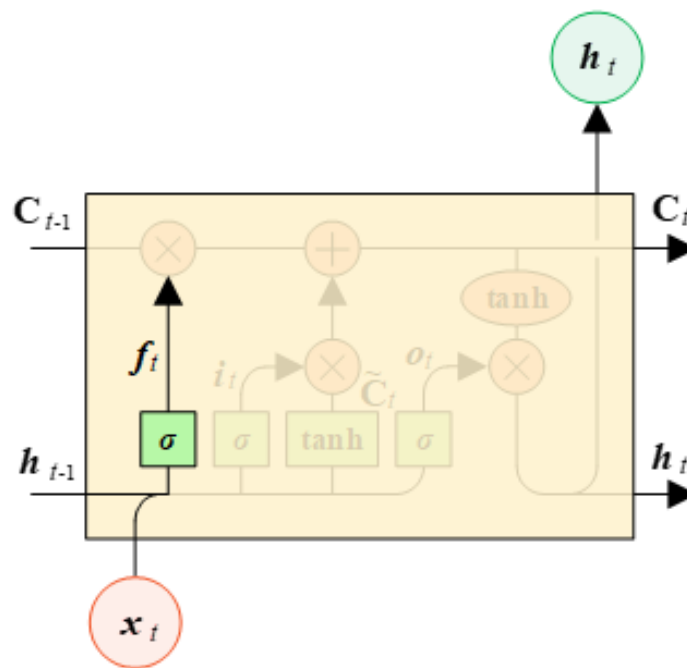
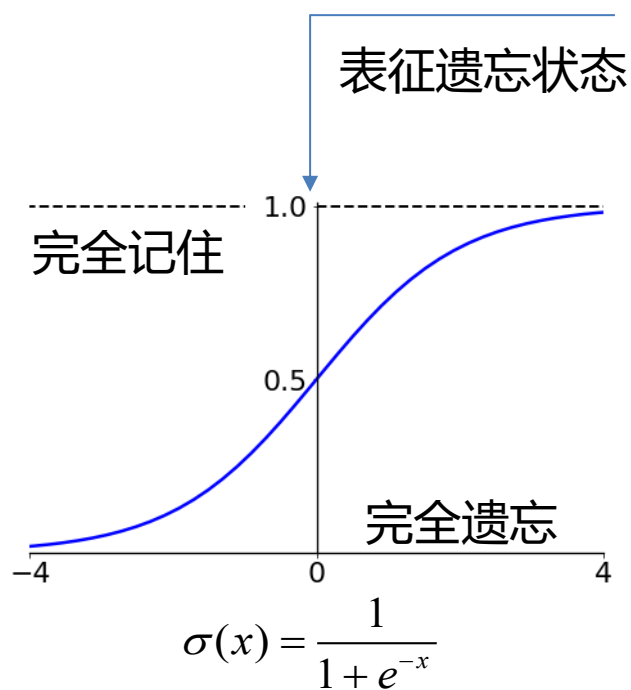


循环神经网络 RNN

□ 长短期记忆模型 (LSTM)

- 遗忘门：决定哪些信息需要从cell state中**抹除**

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



□ 长短期记忆模型 (LSTM)

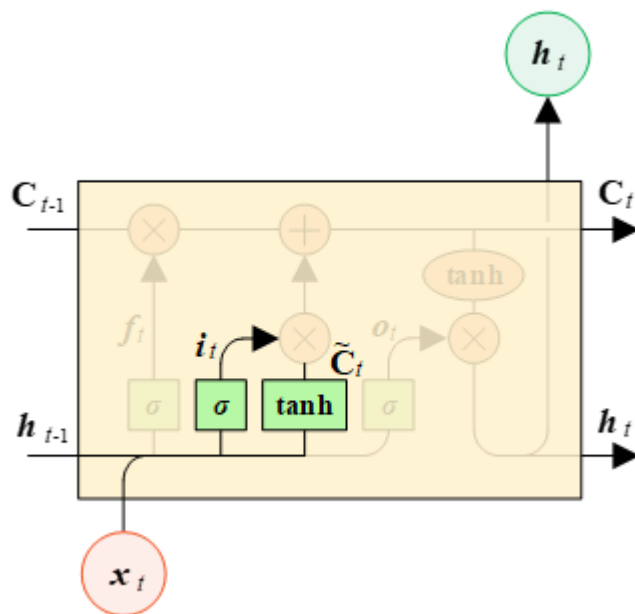
- 输入门：控制**新信息**在细胞状态中的存储

决定细胞状态中的哪些值需要被更新：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

给出可以被加入细胞状态的新信息，作为候选细胞状态：

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$



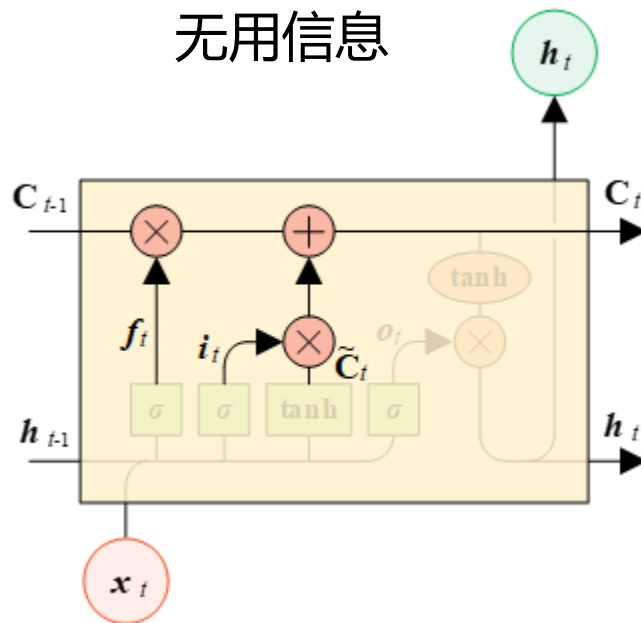
循环神经网络 RNN

□长短期记忆模型 (LSTM)

- **更新**细胞状态中存储的信息 从候选状态中提取有价值信息

$$C_t = f_t \circ C_{t-1} + i_t \circ \tilde{C}_t$$

忘掉之前的
无用信息



◦表示哈德马积



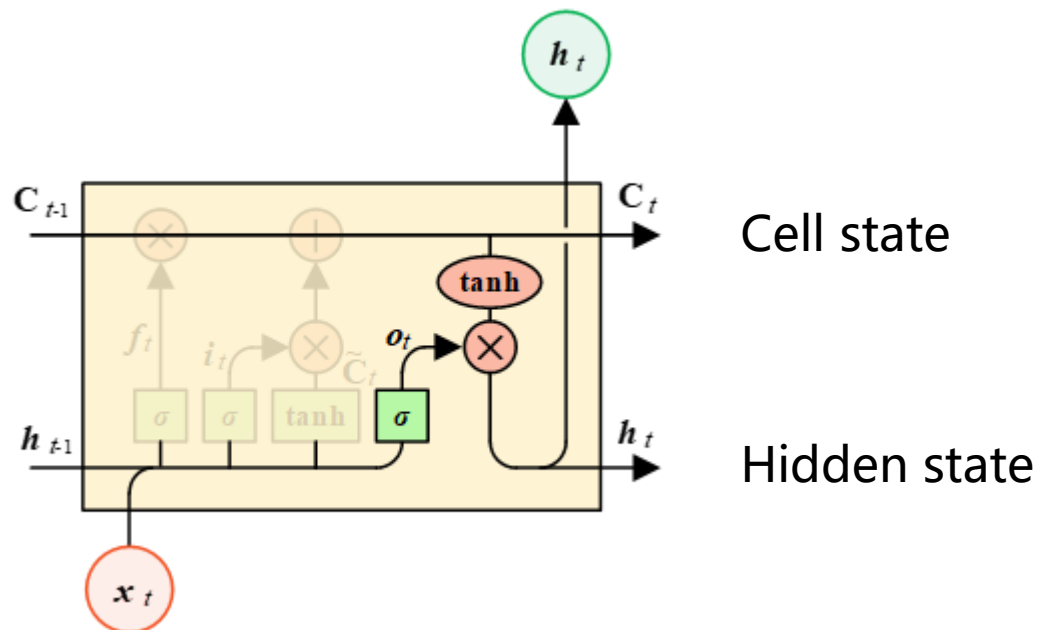
循环神经网络 RNN

□长短期记忆模型 (LSTM)

- 输出门：决定当前时间步**输出**哪些信息 h_t

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \circ \tanh(C_t)$$



◦表示哈德马积



循环神经网络 RNN

□长短期记忆模型 (LSTM) : 总结

LSTM 的前向计算:

一共有 6 个公式

遗忘门 (forget gate)

它决定了上一时刻的单元状态 c_{t-1} 有多少保留到当前时刻 c_t

输入门 (input gate)

它决定了当前时刻网络的输入 x_t 有多少保存到单元状态 c_t

输出门 (output gate)

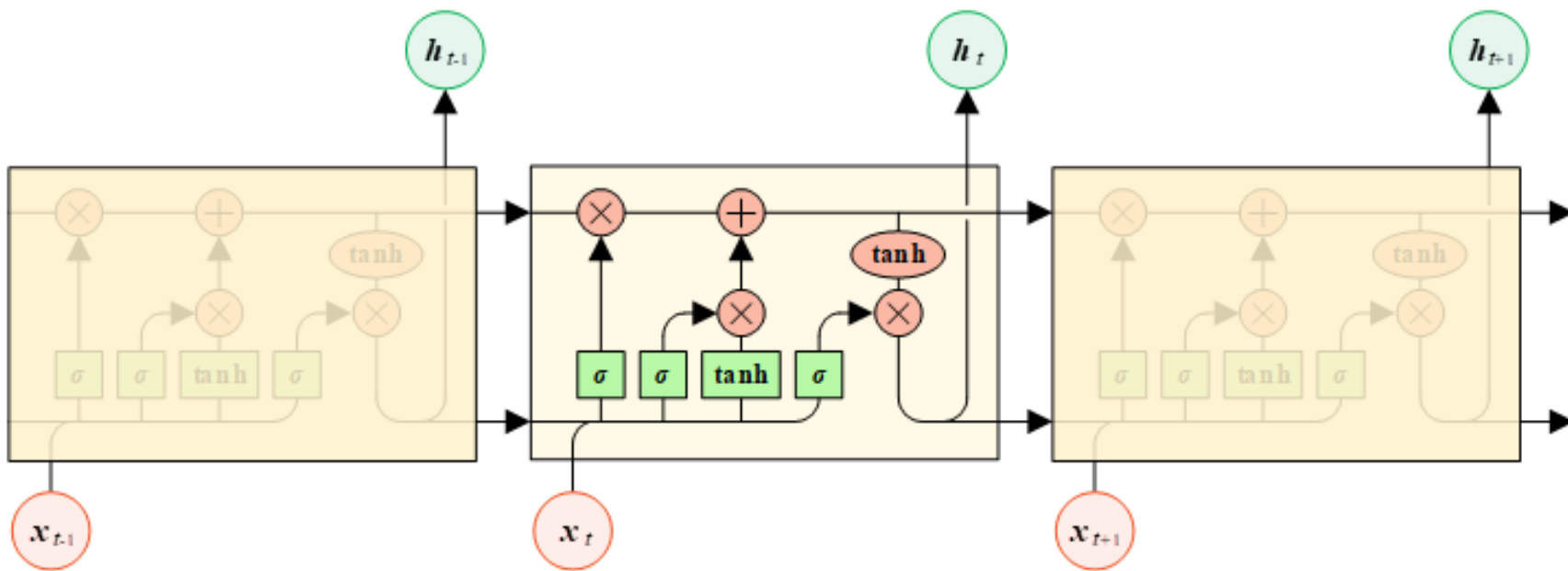
控制单元状态 c_t 有多少输出到 LSTM 的当前输出值 h_t



循环神经网络 RNN

□ 长短期记忆模型 (LSTM)

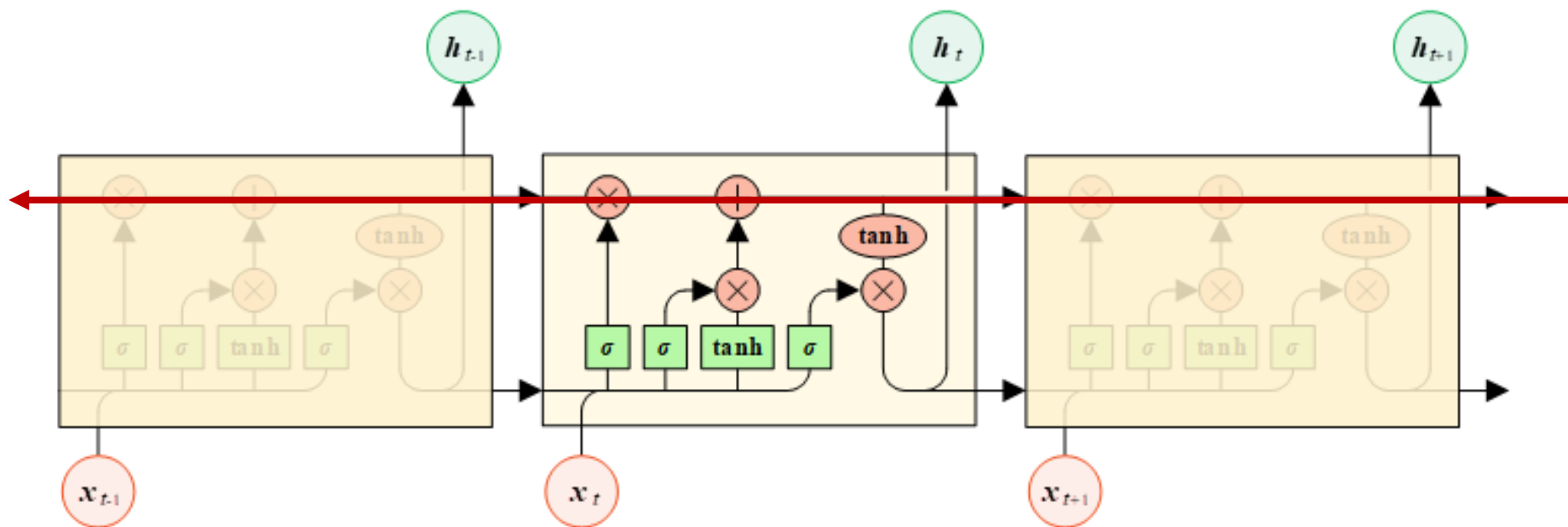
思考： LSTM是如何解决梯度消失/爆炸问题的？



循环神经网络 RNN

□长短期记忆模型 (LSTM)

思考： LSTM是如何解决梯度消失/爆炸问题的？

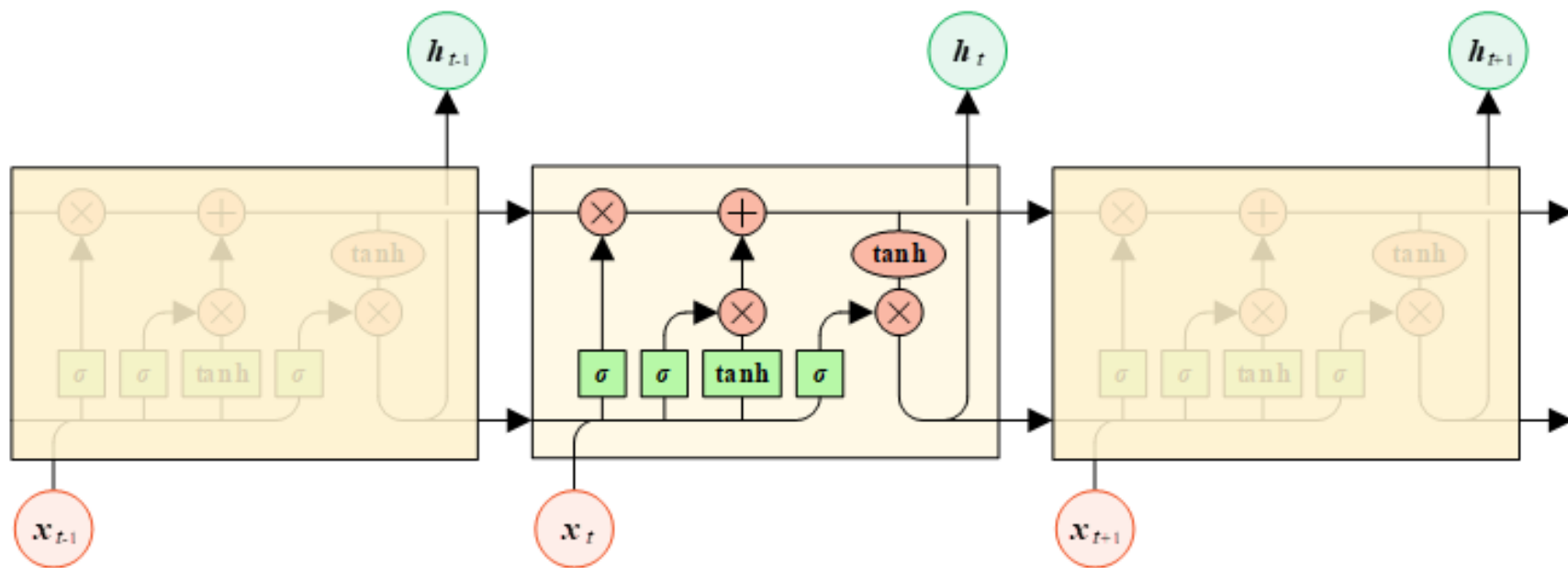


梯度消失问题： LSTM通过直接沿着单元状态（即红色的梯度传递路径）传递梯度，这条路径计算简单，不涉及**指数项**，确保了梯度可以稳定地传递和保留，从而解决梯度消失的问题。

循环神经网络 RNN

□长短期记忆模型 (LSTM)

思考： LSTM是如何解决梯度消失/爆炸问题的？

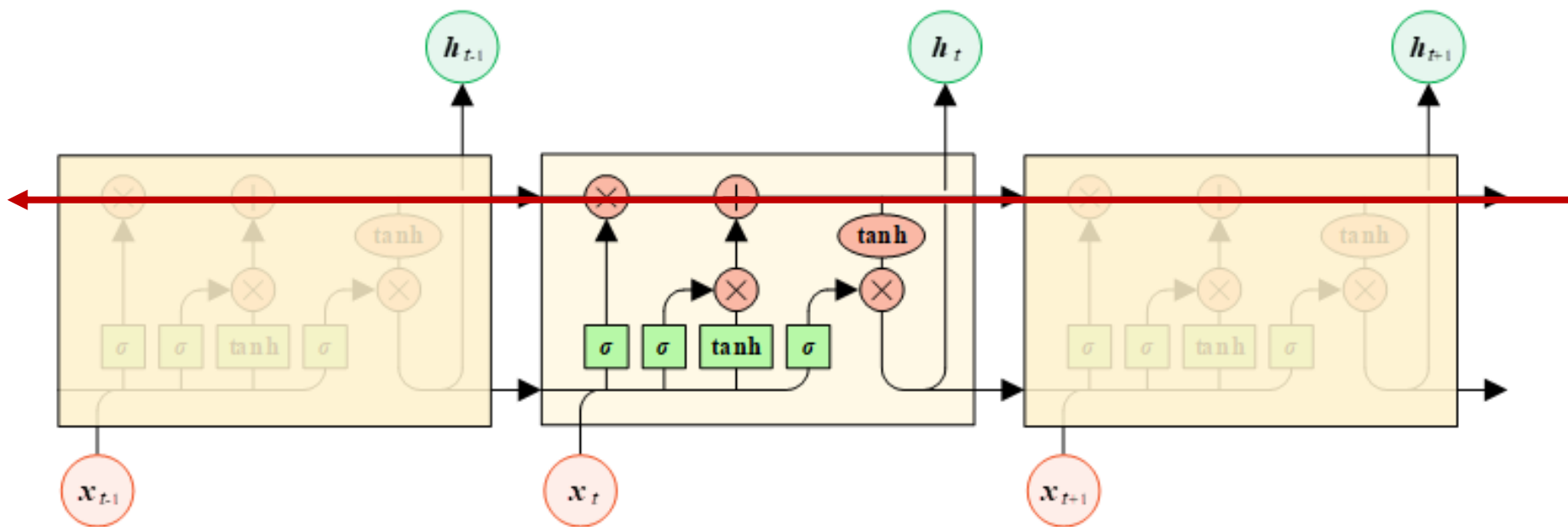


梯度爆炸问题： 由于LSTM门结构（Sigmoid函数）、隐藏状态更新（tanh函数）的引入，梯度在传递时经过门的控制，会被有效地限制在合理的范围内，避免梯度在反向传播过程中不断增大导致的梯度爆炸问题。

循环神经网络 RNN

□ 长短期记忆模型 (LSTM)

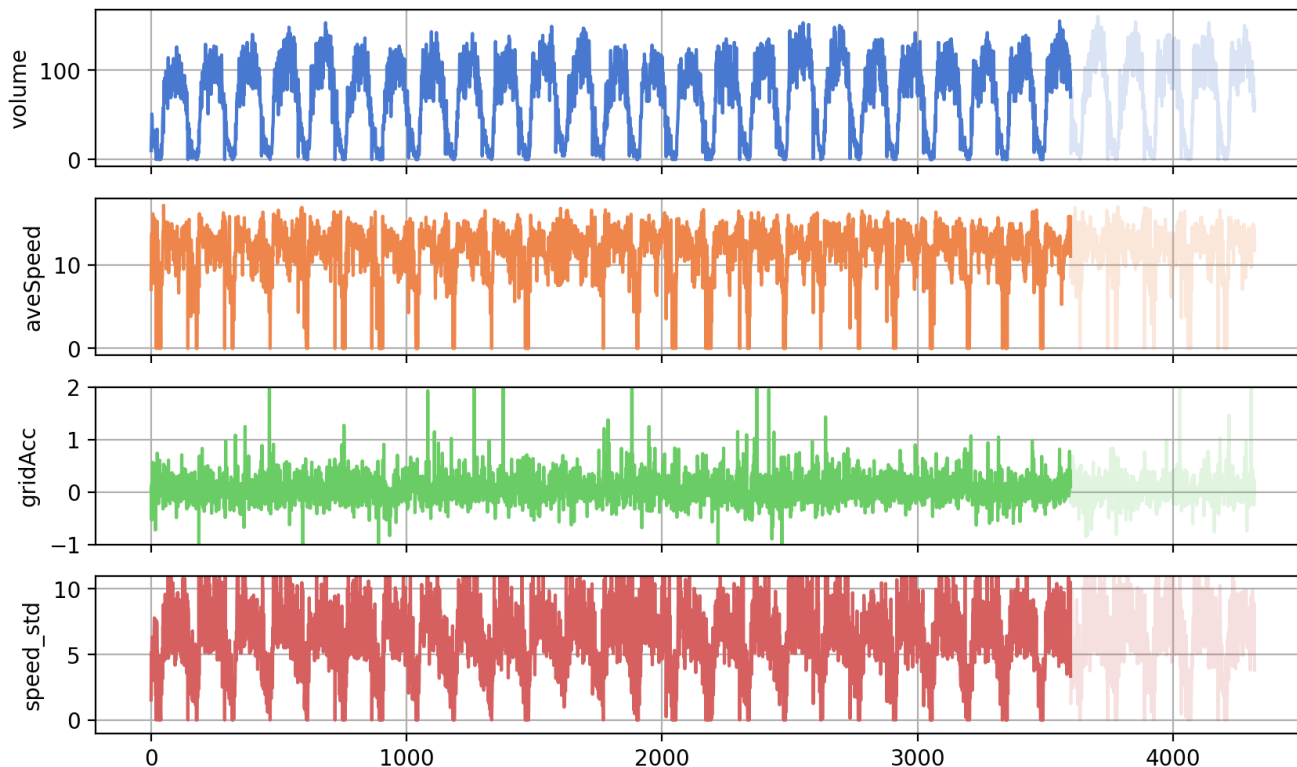
思考： LSTM是如何解决梯度消失/爆炸问题的？



通过门控的机制与梯度直接传递路径的机制，二者共同控制梯度的传导，使得梯度控制在合理的范围内，同时解决梯度消失、爆炸问题

□ 例12-2. 基于RNN的网约车流量预测

和例12-1一样，我们对网约车流量进行预测。但本案例仅关注**一个网格**，以坐标为 (10, 10) 的网格为例，使用LSTM对该网格中的网约车流量进行预测。



各交通流参数变化曲线

❑ 例12-2. 基于RNN的网约车流量预测

● 数据预处理：

- ✓ 集计：案例取**10分钟**为时间粒度对流量进行集计，每天可集计 $1440/10=144$ 个时间步。
- ✓ 数据集划分：取最后5天的数据作为测试集
- ✓ 数据标准化：最大值-最小值标准化

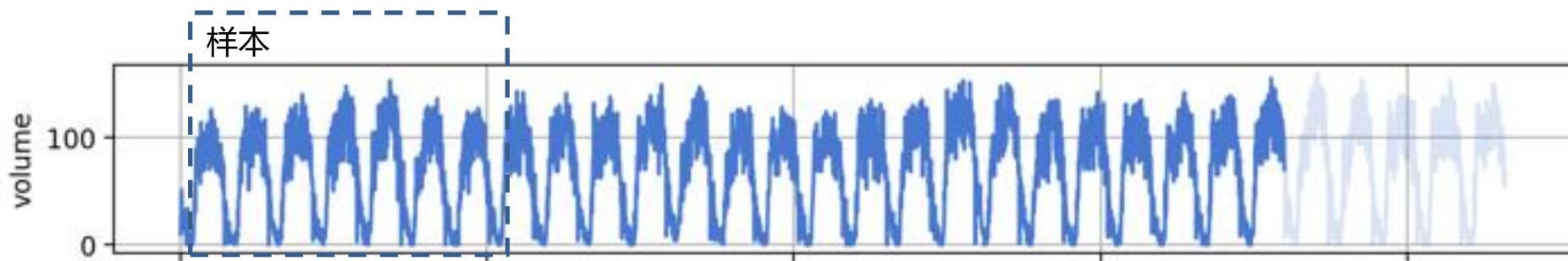
```
from sklearn.preprocessing import MinMaxScaler
# 提取训练集序列、测试集序列
features = ['volume', 'aveSpeed', 'gridAcc', 'speed_std', 'hourid']
train_ts=ts[:-5*144].reset_index(drop=True)[features]
test_ts=ts[-5*144:].reset_index(drop=True)[features]
y_true_test = test_ts.values[..., 0]           # 测试集预测值真值
scaler = MinMaxScaler()                       # 初始化归一化工具
scaler.fit(train_ts.values)                   # 读取数据最值信息
# 归一化数据集
train_ts = pd.DataFrame(scaler.transform(train_ts), columns=train_ts.columns)
test_ts = pd.DataFrame(scaler.transform(test_ts), columns=test_ts.columns)
```


□ 例12-2. 基于RNN的网约车流量预测

● 数据预处理：

✓ 将时序序列转化为独立样本：

在本例中，每个样本包含1009个时间序列值，其中前1008个为输入值（即7天），最后1个为预测标签。（意为：基于过去一周的历史数据预测未来一个时间片的流量情况）



- 数据预处理：将时序序列转化为独立样本：

```
import numpy as np
from sklearn.model_selection import train_test_split
```

将时间序列预测转换为监督学习问题.

```
def ts_to_supervise(ts, window, forecast):
```

```
    """
```

ts: 原始时间序列

window: 输入时间步数量

forecast: 输入时间步数量

```
    """
```

past = [] # 存储输入的时间序列值

future = [] # 存储输出的时间序列值

```
for i in range(len(ts) - window - forecast + 1):
```

```
    past.append(ts[i:i+window])
```

```
    future.append(ts[i+window:i+window+forecast])
```

```
return np.stack(past), np.stack(future)
```

测试集划分

```
ts_x, ts_y = ts_to_supervise(pd.concat([train_ts, test_ts]).values, 7*144, 1)
```

```
train_X, test_X, train_y, test_y = train_test_split(ts_x, ts_y, test_size=len(test_ts), shuffle=False)
```

● 模型构建：基于LSTM的预测模型

```
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import LSTM, Dense, Input, Bidirectional
```

```
# 设定随机种子，保证结果可复现
```

```
SEED = 233
```

```
np.random.seed(SEED)
```

```
tf.random.set_seed(SEED)
```

```
# 循环神经网络模型
```

```
def rnn_model(input_shape):
```

```
    inputs = Input(shape=input_shape)
```

```
    x = LSTM(8)(inputs)
```

```
    x = Dense(8)(x)
```

```
    x = Dense(1)(x)
```

```
    model = Model(inputs=inputs, outputs=x) # 建立模型
```

```
    return model
```

```
model = rnn_model(train_X.shape[1:]) # 实例化模型
```



- 模型训练：损失函数为MSE，使用Adam优化器，学习率为0.001，并设置了早停策略防止严重过拟合训练集数据。

```
from tensorflow.keras import optimizers
from tensorflow.keras.callbacks import EarlyStopping

batch_size = 16                                # 每一训练批次的样本数量
epochs = 50                                    # 最大训练轮数
opt = optimizers.Adam(learning_rate=0.001)     # 优化器 Adam
model.compile(loss='mse', optimizer=opt, metrics=['mae', 'mse'])
# 早停策略
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.001, patience=20)

history = model.fit(
    train_X, train_y[..., 0],
    batch_size=batch_size,
    epochs=epochs,
    validation_split=0.15,                       # 验证集比例
    use_multiprocessing=True,                   # 使用多线程
    callbacks=[early_stopping])
```



● 模型性能评估:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
pred_test = model.predict(test_X) # 预测测试集数据
```

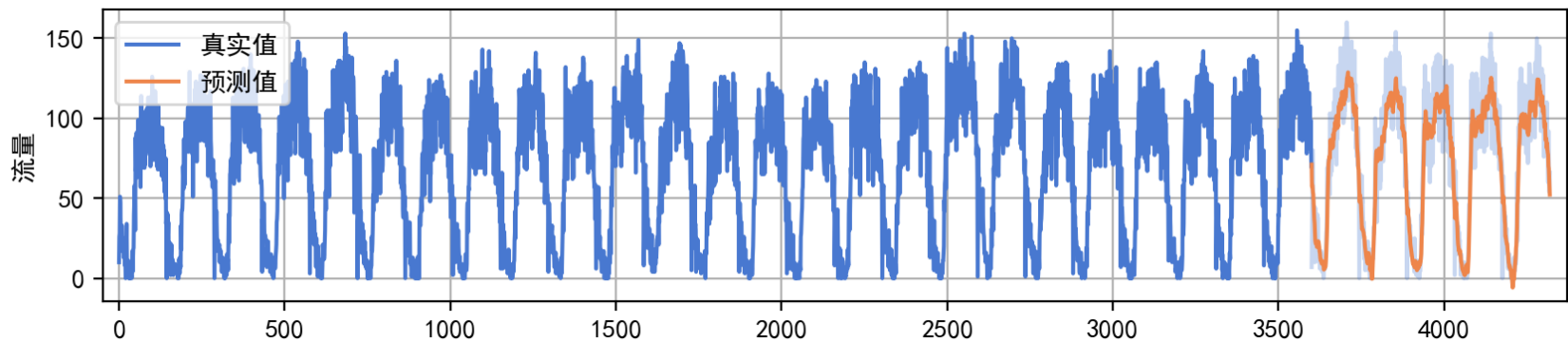
```
inv_test_ts = test_ts.copy()
```

```
inv_test_ts.volume = pred_test
```

```
y_hat_test = scaler.inverse_transform(inv_test_ts)[:, 0] # 还原归一化
```

输出评估指标

```
print(mean_absolute_error(y_true_test, y_hat_test),  
      mean_squared_error(y_true_test, y_hat_test),  
      np.sqrt(mean_squared_error(y_true_test, y_hat_test)))
```



MAE	MSE	RMSE
12.814	278.532	16.690

